

Integer Data Structures y Ordenamiento de Enteros para el Final de EDA

Guia desde cero, orientada a resolver problemas teoricos

Como usar esta guia

Esta guia cubre los PDFs de la carpeta `integer/`. Esta escrita para entender el tema desde cero, con foco en problemas teoricos de examen: definir el modelo, reconocer el problema, escoger la estructura, justificar correctitud y cerrar complejidad.

PDFs cubiertos:

- `eda_slides_06_integer.pdf`: modelos, predecesor, van Emde Boas, X-fast trees, Y-fast trees.
- `Estructuras_de_Datos_Avanzadas___Modelo_de_enteros_II.pdf`: RAM AC0 y Fusion Trees.
- `Estructuras_de_Datos_Avanzadas___Ordenamiento_en_tiempo_lineal.pdf`: ordenamiento de enteros, equivalencia con priority queues y signature sort.

Contents

1	Mapa mental para clasificar problemas de integer	4
1.1	Primera pregunta: el dominio importa?	4
1.2	Segunda pregunta: es sorting o predecesor?	4
1.3	Tercera pregunta: que parametros dominan?	4
1.4	Checklist de respuesta de examen	4
2	Diccionario fundamental desde cero	5
2.1	Palabra de maquina	5
2.2	Universo	5
2.3	Transdichotomous RAM	5
2.4	Word RAM	5
2.5	RAM AC0	5
2.6	Cell probe	5
2.7	Pointer machine	6
2.8	Predecessor y successor	6
2.9	Operaciones del problema predecessor	6
2.10	Por que BST no es suficiente	6
2.11	Trie binario	6
2.12	MSB, LCP y bit critico	6
3	El problema predecessor	7
3.1	Definicion completa	7
3.2	Resultados que debes reconocer	7
3.3	Hint de clasificacion	7

4	van Emde Boas	8
4.1	Idea principal	8
4.2	Campos de un nodo vEB	8
4.3	Por que guardar min y max	8
4.4	Successor en vEB	8
4.5	Correctitud	8
4.6	Complejidad	9
4.7	Problema de espacio	9
4.8	Como escribir vEB en examen	9
5	X-fast trees	10
5.1	Motivacion	10
5.2	Representacion	10
5.3	Query de successor/predecesor	10
5.4	Complejidad	10
5.5	Cuando mencionar X-fast	10
6	Y-fast trees	11
6.1	Idea principal	11
6.2	Indirection	11
6.3	Query	11
6.4	Complejidad	11
6.5	Correctitud	11
7	Fusion Trees	12
7.1	Que problema resuelven	12
7.2	Idea de alto nivel	12
7.3	El problema dentro de un nodo	12
7.4	Trie conceptual y bits importantes	12
7.5	Sketch	12
7.6	Cuidado: sketch de query no basta	12
7.7	Consulta dentro de un nodo	13
7.8	Por que cabe en una palabra	13
7.9	Complejidad	13
7.10	RAM AC0 y multiplicacion	13
7.11	Plantilla de examen para Fusion Trees	13
8	Ordenamiento de enteros	14
8.1	Por que se puede superar $O(n \log n)$	14
8.2	Counting sort	14
8.3	Radix sort	14
8.4	Tabla de resultados	14
8.5	Equivalencia sorting - priority queue	14
9	Packed sort	15
9.1	Idea	15
9.2	Por que importa	15
9.3	Hint de examen	15
10	Signature sort	16
10.1	Objetivo	16
10.2	Condicion base	16
10.3	Paso 1: partir en digitos	16
10.4	Paso 2: hashing perfecto de digitos	16
10.5	Paso 3: signatures compactas	16
10.6	Paso 4: trie comprimido de signatures	16
10.7	Paso 5: ordenar aristas recursivamente	16
10.8	Paso 6: permutar hijos	17

10.9 Paso 7: inorder traversal	17
10.10Correctitud	17
10.11Complejidad	17
10.12Que memorizar para examen	17
11 Como reconocer que estructura usar	18
12 Plantillas de solucion teorica	18
12.1 Plantilla vEB	18
12.2 Plantilla Y-fast	18
12.3 Plantilla Fusion Tree	18
12.4 Plantilla radix/counting	19
13 Mapa de slides cubiertos	20
14 Apendice: mapa pagina por pagina de los slides de integer	21

1 Mapa mental para clasificar problemas de integer

1.1 Primera pregunta: el dominio importa?

En el modelo de comparaciones, solo puedes preguntar si $a < b$. En integer structures puedes usar que las llaves son enteros de w bits:

$$U = \{0, 1, \dots, 2^w - 1\}.$$

Eso permite usar bits, tries, hashing, particion por digitos y operaciones word-RAM. La primera pregunta de clasificacion es:

El problema solo compara elementos, o explota que son enteros de rango acotado?

1.2 Segunda pregunta: es sorting o predecessor?

Problema	Frases tipicas	Herramientas
Predecessor/successor	mayor menor que x , siguiente, anterior	vEB, X-fast, Y-fast, Fusion Tree
Diccionario dinamico	insert/delete/member	hashing, tries, integer dictionaries
Ordenamiento	ordenar n enteros de w bits	counting, radix, signature sort
Priority queue	insert, find-min, delete-min	equivalencia con sorting
Busqueda por bits	prefijos, trie binario, universe	vEB, X-fast, Fusion Tree sketches

1.3 Tercera pregunta: que parametros dominan?

En integer no basta decir $O(\log n)$. Debes mirar:

- n : cantidad de llaves almacenadas.
- w : bits por palabra/llave.
- $u = 2^w$: tamano del universo.
- espacio: $\Theta(u)$ puede ser imposible si u es enorme.
- modelo: word RAM, AC0 RAM, cell probe, pointer machine.

1.4 Checklist de respuesta de examen

1. Define el universo U y el tamano de palabra w .
2. Di si el modelo permite operaciones bitwise, shifts, multiplicacion o no.
3. Identifica si se necesita predecessor, successor, insert/delete o sorting.
4. Escoge estructura: vEB, X-fast, Y-fast, Fusion Tree, radix/signature sort.
5. Explica el invariante: clusters, summaries, hash tables, sketches, trie.
6. Justifica complejidad en funcion de n, w, u .
7. Menciona el trade-off de espacio si aplica.

2 Diccionario fundamental desde cero

2.1 Palabra de maquina

Una palabra de maquina es un bloque de w bits que la RAM puede manejar en $O(1)$ tiempo. Si $w = 64$, una palabra puede guardar enteros entre 0 y $2^{64} - 1$.

En teoria se deja w como parametro. Muchas complejidades se escriben como $O(\log w)$, $O(\log_w n)$ o $O(\log \log u)$.

2.2 Universo

El universo U es el conjunto de posibles llaves:

$$U = \{0, 1, \dots, u - 1\}.$$

Si las llaves tienen w bits:

$$u = 2^w, \quad w = \log_2 u.$$

2.3 Transdichotomous RAM

El modelo transdichotomous RAM une dos mundos: el tamaño de palabra depende del tamaño del problema. Normalmente se asume:

$$w \geq \log S,$$

donde S es el espacio usado o el tamaño del input. Esto permite indexar la memoria con una palabra.

2.4 Word RAM

Word RAM es el modelo donde una palabra de w bits permite operaciones estilo C en $O(1)$:

- suma/resta;
- comparacion;
- bitwise AND/OR/XOR;
- shifts;
- a veces multiplicacion, dependiendo del submodelo.

Integer data structures explotan estas operaciones para superar barreras del modelo de comparaciones.

2.5 RAM AC0

RAM AC0 permite operaciones realizables por circuitos de profundidad constante con fan-in/fan-out no acotado. En los slides se enfatiza:

- operaciones bitwise si se consideran $O(1)$;
- multiplicacion no esta permitida por definicion;
- es un modelo mas restringido que word RAM con multiplicacion.

Esto importa porque algunos trucos de Fusion Trees usan multiplicacion para comprimir bits. Si el modelo no la permite, necesitas variantes mas cuidadosas.

2.6 Cell probe

Cell probe cuenta solo lecturas y escrituras de memoria. La computacion interna es gratis. Sirve para lower bounds: si incluso con computacion gratis necesitas muchas probes, entonces ningun algoritmo real puede ser mas rapido.

2.7 Pointer machine

En pointer machine no puedes indexar memoria arbitrariamente como en word RAM. Sigues punteros entre nodos. Es un modelo mas debil para integer tricks porque no puedes usar aritmetica de bits de forma tan agresiva.

2.8 Predecessor y successor

Dado un conjunto dinamico $S \subseteq U$:

$$\begin{aligned}\text{pred}(x) &= \max\{y \in S : y < x\}, \\ \text{succ}(x) &= \min\{y \in S : y > x\}.\end{aligned}$$

Nota: algunos slides tienen un typo en successor como max; conceptualmente successor es el minimo elemento mayor que x .

2.9 Operaciones del problema predecessor

- `insert(x)`: agrega x .
- `delete(x)`: elimina x .
- `predecessor(x)`: mayor llave menor que x .
- `successor(x)`: menor llave mayor que x .
- `member(x)`: saber si x pertenece a S .

2.10 Por que BST no es suficiente

Un BST balanceado da $O(\log n)$ por operacion y es optimo en el modelo de comparaciones. Pero si las llaves son enteros de w bits, podemos usar su representacion binaria y obtener tiempos como:

$$O(\log w), \quad O(\log_w n), \quad O(\sqrt{\log n}).$$

2.11 Trie binario

Un trie binario guarda llaves segun sus bits. Cada nivel mira un bit. Una llave de w bits tiene profundidad w . Buscar predecessor en un trie completo seria $O(w)$, pero vEB, X-fast y Fusion Trees comprimen o aceleran esta idea.

2.12 MSB, LCP y bit critico

Para dos enteros x, y , el bit mas significativo donde difieren determina su orden. Si comparten un prefijo largo, estan cerca en el trie. Fusion Trees usan solo ciertos bits criticos para distinguir muchas llaves en paralelo.

3 El problema predecesor

3.1 Definicion completa

Mantienes un conjunto S de n enteros de w bits. Debes responder predecesor y successor bajo updates. Este problema es central porque muchas estructuras se reducen a el:

- diccionarios ordenados;
- priority queues;
- busqueda por rangos;
- ordenamiento de enteros;
- estructuras de indices.

3.2 Resultados que debes reconocer

Estructura	Tiempo	Espacio
BST comparaciones	$O(\log n)$	$\Theta(n)$
van Emde Boas	$O(\log w) = O(\log \log u)$	$\Theta(u)$
X-fast tree	$O(\log w)$ query, $\Theta(w)$ update	$\Theta(nw)$
Y-fast tree	$O(\log w)$ insert/delete/amortizado	$\Theta(n)$
Fusion tree	$O(\log_w n)$	$\Theta(n)$

3.3 Hint de clasificacion

Si el enunciado dice “dado x , encuentra el elemento mas cercano menor/mayor” o “mantener conjunto ordenado de enteros”, piensa en predecesor. Si solo necesitas membership, hashing basta. Si necesitas orden, necesitas predecesor o BST/integer tree.

4 van Emde Boas

4.1 Idea principal

van Emde Boas divide el universo en una matriz de $\sqrt{u}$ clusters, cada uno de tamaño $\sqrt{u}$. Un entero x se escribe como:

$$x = \langle c, i \rangle,$$

donde:

- c es el número de cluster;
- i es la posición dentro del cluster.

Si $u = 2^w$, partir en $\sqrt{u}$ equivale a partir los bits de x en mitad alta y mitad baja.

4.2 Campos de un nodo vEB

Un nodo vEB V de universo u guarda:

- $V.min$: mínimo presente.
- $V.max$: máximo presente.
- $V.cluster[c]$: vEB del cluster c .
- $V.summary$: vEB que guarda que clusters no están vacíos.

4.3 Por qué guardar min y max

min y max permiten resolver casos extremos en $O(1)$:

- si $x < V.min$, successor es $V.min$;
- si $x > V.max$, no hay successor;
- insertar en estructura vacía es inmediato.

Además, evitan tener que bajar recursivamente para conocer extremos.

4.4 Successor en vEB

Para $\text{succ}(V, x = \langle c, i \rangle)$:

1. Si $x < V.min$, responde $V.min$.
2. Si el cluster c existe y $i < V.cluster[c].max$, el successor está dentro del mismo cluster:

$$\langle c, \text{succ}(V.cluster[c], i) \rangle.$$

3. Si no, busca el siguiente cluster no vacío:

$$c' = \text{succ}(V.summary, c).$$

4. La respuesta es el mínimo de ese cluster:

$$\langle c', V.cluster[c'].min \rangle.$$

4.5 Correctitud

Hay dos casos. Si existe un elemento mayor que x dentro del mismo cluster, ese elemento es necesariamente menor que cualquier elemento de un cluster posterior, por lo que el successor está ahí. Si no existe, todo successor debe estar en un cluster con índice mayor a c ; el primer cluster no vacío da el menor elemento posible, usando su mínimo.

4.6 Complejidad

Cada llamada reduce el universo de u a $\sqrt{u}$:

$$T(u) = T(\sqrt{u}) + O(1).$$

Si $u = 2^w$, tras una llamada el numero de bits pasa de w a $w/2$. Entonces:

$$T(w) = T(w/2) + O(1) = O(\log w) = O(\log \log u).$$

4.7 Problema de espacio

Un vEB completo crea clusters aunque esten vacios:

$$\Theta(u) \text{ espacio.}$$

Esto es demasiado si el universo es enorme y n es pequeno. Por eso aparecen X-fast/Y-fast y la idea de no almacenar vacios.

4.8 Como escribir vEB en examen

1. Divido el universo en high/low de tamano $\sqrt{u}$.
2. Cada nodo guarda min, max, clusters y summary.
3. El summary guarda que clusters estan no vacios.
4. Successor mira primero mismo cluster; si falla, usa summary para saltar al siguiente cluster.
5. La recurrencia es $T(u) = T(\sqrt{u}) + O(1)$.
6. Tiempo $O(\log \log u)$, espacio naive $\Theta(u)$.

5 X-fast trees

5.1 Motivacion

vEB es rapido pero usa demasiado espacio. X-fast trees parten de una idea de trie binario: no almacenar ceros/vacios. Solo se guardan prefijos que existen en alguna llave.

5.2 Representacion

Para cada nivel del trie, se guarda una hash table con los prefijos existentes de ese nivel. Si una llave tiene w bits, aporta un prefijo por nivel:

$$\Theta(w) \text{ prefijos por llave.}$$

Por eso el espacio es:

$$\Theta(nw).$$

5.3 Query de successor/predecessor

Para buscar x :

1. Haces binary search sobre niveles del trie.
2. En cada nivel preguntas en la hash table si el prefijo de x existe.
3. Encuentras el nodo mas profundo del trie que coincide con un prefijo de x .
4. Desde el punto donde la busqueda falla, usas punteros a hojas/predecessor o successor para responder.

5.4 Complejidad

Hay w niveles. Binary search sobre niveles cuesta:

$$O(\log w).$$

Cada prueba de prefijo es hash en $O(1)$ esperado. Updates insertan o borran prefijos en todos los niveles:

$$\Theta(w).$$

5.5 Cuando mencionar X-fast

X-fast es importante como paso intermedio hacia Y-fast. Si el problema pregunta “como ahorrar espacio de vEB”, menciona que no guardas clusters vacios y usas hash tables de prefijos, pero aun queda $\Theta(nw)$ espacio.

6 Y-fast trees

6.1 Idea principal

Y-fast tree = X-fast tree + indirection.

En lugar de guardar todas las llaves en un X-fast, guardas representantes de grupos. Cada grupo tiene tamaño esperado $\Theta(w)$ y se maneja con una estructura local, por ejemplo BST balanceado.

6.2 Indirection

Indirection significa partir el conjunto grande en chunks pequeños:

$$n \text{ elementos} \rightarrow \frac{n}{w} \text{ grupos de tamaño } \Theta(w).$$

El X-fast se usa solo sobre representantes de grupos, no sobre todos los elementos.

6.3 Query

Para predecessor/successor de x :

1. Usa X-fast sobre representantes para ubicar el grupo candidato.
2. Busca dentro del grupo local.
3. Si hace falta, revisa grupo vecino.

6.4 Complejidad

X-fast sobre representantes:

$$O(\log w).$$

Busqueda local en grupo de tamaño $\Theta(w)$:

$$O(\log w).$$

Entonces:

$$O(\log w).$$

El espacio baja a:

$$\Theta(n).$$

Updates son $O(\log w)$ amortizado/esperado porque de vez en cuando se parte o fusiona un grupo.

6.5 Correctitud

Los representantes ordenan los grupos y cada grupo cubre un intervalo de llaves. El predecessor de x solo puede estar en el grupo donde caería x o en el grupo anterior. Como X-fast localiza ese vecindario y la estructura local resuelve dentro del grupo, la respuesta es correcta.

7 Fusion Trees

7.1 Que problema resuelven

Fusion Trees resuelven predecessor/successor en:

$$O(\log_w n)$$

tiempo con espacio lineal, en word RAM suficientemente fuerte. El slide lo presenta como:

$$O(\log_w n)$$

y lo compara con vEB/Y-fast:

$$\min\{O(\log w), O(\log_w n)\}.$$

7.2 Idea de alto nivel

Un BST normal tiene branching factor 2, por eso altura $\log n$. Un Fusion Tree usa branching factor:

$$B = w^{1/5}.$$

Si cada nodo puede decidir en $O(1)$ entre B hijos, la altura es:

$$\log_B n = \log_{w^{1/5}} n = O(\log_w n).$$

7.3 El problema dentro de un nodo

Un nodo guarda:

$$x_0 < x_1 < \dots < x_{k-1}, \quad k = O(w^{1/5}).$$

Para bajar al hijo correcto, dado q necesitas encontrar predecessor/successor entre esas k llaves en $O(1)$. No puedes comparar una por una.

7.4 Trie conceptual y bits importantes

Imagina un trie binario con las k llaves del nodo. Para distinguir k llaves ordenadas, no necesitas todos los w bits. Solo necesitas los niveles donde el trie se ramifica. Hay a lo mucho:

$$k - 1$$

nodos internos relevantes, por tanto $O(k)$ bits importantes.

7.5 Sketch

El sketch de una llave x extrae esos bits importantes:

$$\text{Sketch}(x) = \text{bits de } x \text{ en posiciones criticas.}$$

Para las llaves guardadas en el nodo:

$$\text{Sketch}(x_0) < \text{Sketch}(x_1) < \dots < \text{Sketch}(x_{k-1}).$$

Esto preserva el orden entre las llaves almacenadas.

7.6 Cuidado: sketch de query no basta

Aunque los sketches preservan el orden de las llaves, $\text{Sketch}(q)$ para una query arbitraria puede caer en una posición que no corresponde al predecessor real. Por eso se necesita corregir usando el longest common prefix entre q y las llaves vecinas candidatas.

7.7 Consulta dentro de un nodo

Conceptualmente:

1. Calcula $\text{Sketch}(q)$.
2. Compara en paralelo contra los sketches de las llaves del nodo.
3. Obtiene candidatos vecinos x_i, x_{i+1} .
4. Usa LCP/bit critico para decidir donde falla la busqueda real en el trie.
5. Devuelve el hijo correcto o la llave predecessor/successor local.

7.8 Por que cabe en una palabra

Con $k = O(w^{1/5})$, cada sketch tiene $O(k)$ bits. Todos los sketches juntos usan:

$$O(k^2) = O(w^{2/5})$$

bits, que cabe en $O(1)$ palabras. Por eso se pueden comparar en paralelo con word tricks.

7.9 Complejidad

Cada nodo se procesa en $O(1)$. La altura del arbol es:

$$O(\log_w n).$$

Entonces predecessor/successor cuesta:

$$O(\log_w n).$$

7.10 RAM AC0 y multiplicacion

Las versiones classicas usan multiplicacion para empaquetar/extraer bits. En RAM AC0 la multiplicacion no esta permitida. Por eso el slide separa el modelo: debes saber que no todas las operaciones word-RAM son gratis en todos los modelos.

7.11 Plantilla de examen para Fusion Trees

1. Construyo un B-tree con branching factor $w^{1/5}$.
2. En cada nodo guardo $k = O(w^{1/5})$ llaves.
3. Uso bits criticos del trie virtual para formar sketches.
4. Los sketches de las llaves preservan orden.
5. Comparo sketches en paralelo para encontrar candidato en $O(1)$.
6. Corrijo con LCP/bit de falla para queries que no son llaves.
7. Altura $O(\log_w n)$, espacio lineal.

8 Ordenamiento de enteros

8.1 Por que se puede superar $O(n \log n)$

El lower bound $\Omega(n \log n)$ aplica al modelo de comparaciones. Si las llaves son enteros de w bits, puedes mirar digitos/bits directamente. Por eso counting sort, radix sort y algoritmos avanzados pueden ser lineales bajo condiciones sobre w .

8.2 Counting sort

Counting sort cuenta cuantas veces aparece cada valor del universo. Para llaves en $[0, 2^w)$:

$$O(n + 2^w).$$

Es lineal si:

$$2^w = O(n) \iff w = O(\log n).$$

8.3 Radix sort

Radix sort divide cada entero en digitos y ordena por digito usando counting sort estable. Si cada digito tiene $\Theta(\log n)$ bits, hay $n^{\Theta(1)}$ posibles valores por digito y cada pasada cuesta $O(n)$.

Numero de pasadas:

$$\Theta\left(\frac{w}{\log n}\right).$$

Complejidad:

$$O\left(n \frac{w}{\log n}\right).$$

Es lineal si:

$$w = O(\log n).$$

8.4 Tabla de resultados

Metodo	Complejidad	Condicion/nota
Comparaciones	$O(n \log n)$	sin condiciones
Counting sort	$O(n + 2^w)$	lineal si $w = O(\log n)$
Radix sort	$O(nw / \log n)$	lineal si $w = O(\log n)$
vEB sorting	$O(n \log w)$	insertar/extract predecessor
Signature sort	$O(n)$	bajo word-size grande
Han 2004	$O(n \log \log n)$	RAM AC0
Han-Thorup	aleatorizado sublog	resultado avanzado

8.5 Equivalencia sorting - priority queue

Si tienes una priority queue con operaciones en $O(f(n, w))$, puedes ordenar:

1. Insertas los n elementos.
2. Extraes minimos en orden.

Costo:

$$O(nf(n, w)).$$

En sentido inverso, si tienes sorting en $O(T(n, w))$, puedes construir priority queues con costo amortizado relacionado:

$$O(T(n, w)/n).$$

El slide menciona Quickheap como ejemplo conceptual.

9 Packed sort

9.1 Idea

Packed sort aprovecha que muchos enteros pequenos caben en una palabra. Si cada entero tiene b bits y:

$$w = \Omega(b \log n \log \log n),$$

entonces se pueden ordenar n enteros de b bits en $O(n)$.

9.2 Por que importa

Signature sort reduce el problema original a ordenar firmas/ hashes de pocos bits. Cuando esos valores cumplen la condicion de packed sort, el caso base se resuelve en tiempo lineal.

9.3 Hint de examen

No necesitas implementar packed sort. Debes saber su rol: es el caso base lineal cuando los enteros ya fueron reducidos a suficientes pocos bits por recursion/signatures.

10 Signature sort

10.1 Objetivo

Signature sort busca ordenar n enteros de w bits en tiempo lineal bajo una condicion sobre w , usando hashing, tries comprimidos y packed sort.

10.2 Condicion base

Los slides asumen convenientemente:

$$w \geq \log^{2+\epsilon} n \log \log n.$$

La idea es tener palabras suficientemente grandes para empaquetar y manipular digitos.

10.3 Paso 1: partir en digitos

Particionas cada entero en:

$$\log^\epsilon n$$

bloques/digitos de igual tamano. Cada digito tiene al menos $\log^2 n$ bits bajo la condicion asumida.

10.4 Paso 2: hashing perfecto de digitos

Aunque el universo de cada digito es enorme, solo aparecen:

$$n \log^\epsilon n$$

digitos en el input. Se reemplaza cada digito por un hash perfecto de $O(\log n)$ bits.

Esto preserva identidad: si dos digitos eran iguales, tienen la misma firma; si eran distintos, no colisionan. Pero no preserva orden. Este detalle es crucial.

10.5 Paso 3: signatures compactas

Cada entero ahora se representa por $\log^\epsilon n$ firmas de $O(\log n)$ bits. El total por entero es:

$$O(\log^{1+\epsilon} n).$$

Ahora los objetos son mucho mas pequenos que los enteros originales.

10.6 Paso 4: trie comprimido de signatures

Construyes un trie sobre las signatures. Luego comprimes nodos que no ramifican. El trie comprimido tiene:

$$O(n)$$

nodos, no $O(n \log^\epsilon n)$.

El inoder del trie ordena por signatures, pero todavia no necesariamente por valores originales, porque el hashing no preserva orden de digitos.

10.7 Paso 5: ordenar aristas recursivamente

Para corregir el orden real, ordenas las aristas del trie por tuplas:

(indice del nodo, primer digito original, indice del hijo).

Esto reduce la cantidad de bits relevantes en cada nivel recursivo. Despues de una cantidad constante de niveles dependiente de ϵ , llegas a enteros de:

$$b = O\left(\frac{w}{\log n \log \log n}\right)$$

bits, donde packed sort aplica en $O(n)$.

10.8 Paso 6: permutar hijos

Una vez ordenadas las aristas segun el valor original, permutas los hijos de cada nodo del trie comprimido en ese orden.

10.9 Paso 7: inorder traversal

Con los hijos ya ordenados correctamente, el inorder traversal del trie da los enteros originales ordenados.

10.10 Correctitud

El trie agrupa enteros por secuencias de digitos. El hashing permite construir la estructura compacta sin perder igualdad de digitos. Como el hashing no preserva orden, el algoritmo corrige el orden de las aristas usando los digitos originales. Una vez que cada conjunto de hijos esta ordenado por el digito real correspondiente, recorrer el trie en orden produce exactamente el orden lexicografico de los digitos, que coincide con el orden numerico de enteros.

10.11 Complejidad

Cada fase procesa $O(n)$ objetos por nivel. La recursion tiene cantidad constante de niveles para ϵ fijo y el caso base packed sort es lineal. Por tanto:

$$O(n).$$

10.12 Que memorizar para examen

No memorices detalles de implementacion. Memoriza la historia:

1. dividir enteros en digitos;
2. hash perfecto para reducir representacion;
3. construir trie comprimido;
4. ordenar aristas recursivamente usando digitos originales;
5. usar packed sort como caso base;
6. inorder traversal da el orden final.

11 Como reconocer que estructura usar

Frase del problema	Usa	Razon
Predecesor con universo pequeno	vEB	$O(\log \log u)$, espacio universo
Predecesor con espacio lineal	Y-fast tree	vEB-like sin $\Theta(u)$ espacio
Predecesor teorico rapido en word RAM	Fusion tree	branching alto con sketches
Solo membership	Hashing	no necesitas orden
Ordenar enteros con $w = O(\log n)$	radix/counting	tiempo lineal
Ordenar enteros grandes teorico	signature sort	reduce bits y usa packed sort
Priority queue y sorting	equivalencia sorting-PQ	reducciones entre ambos
Modelo sin multiplicacion	RAM AC0	cuidado con word tricks

12 Plantillas de solucion teorica

12.1 Plantilla vEB

1. Universo $u = 2^w$.
2. Represento x como $(high(x), low(x))$.
3. Cada nodo guarda min, max, clusters y summary.
4. Para successor, primero intento mismo cluster; si falla, busco siguiente cluster no vacio en summary.
5. Correctitud por separacion lexicografica de cluster y posicion interna.
6. Recurrencia $T(u) = T(\sqrt{u}) + O(1)$, entonces $O(\log \log u)$.
7. Espacio naive $\Theta(u)$.

12.2 Plantilla Y-fast

1. Agrupo elementos en buckets de tamano $\Theta(w)$.
2. Guardo representantes en un X-fast tree.
3. Cada bucket tiene BST local.
4. Query localiza bucket candidato con X-fast y busca localmente.
5. Updates parten/fusionan buckets para mantener tamano.
6. Tiempo $O(\log w)$ esperado/amortizado, espacio $\Theta(n)$.

12.3 Plantilla Fusion Tree

1. Uso B-tree de grado $w^{1/5}$.
2. En cada nodo debo decidir entre $w^{1/5}$ hijos en $O(1)$.
3. Tomo bits criticos del trie virtual y construyo sketches.
4. Comparo sketches empaquetados en paralelo.
5. Corrijo candidato con LCP/bit de falla.
6. Altura $O(\log_w n)$, espacio lineal.

12.4 Plantilla radix/counting

1. Si universo pequeño, counting sort cuesta $O(n + u)$.
2. Si $u = 2^w$ y $w = O(\log n)$, counting puede ser lineal.
3. Para radix, parto cada entero en dígitos de $\Theta(\log n)$ bits.
4. Cada pasada usa counting sort estable.
5. Número de pasadas $\Theta(w/\log n)$.
6. Costo $O(nw/\log n)$.

13 Mapa de slides cubiertos

eda_slides_06_integer.pdf

- Paginas 1–5: objetivos del bloque Integer I.
- Paginas 6–11: modelos: word, transdichotomous RAM, word RAM, cell probe, pointer machine.
- Paginas 12–17: predecessor problem y tabla de resultados: BST, vEB, Y-fast, Fusion Trees y lower bounds.
- Paginas 18–26: van Emde Boas, coordenadas jerarquicas, clusters, summary, min/max y successor.
- Paginas 27–30: tree view e indirection.
- Paginas 31–35: ahorro de espacio en vEB.
- Paginas 36–40: X-fast trees.
- Paginas 41–45: Y-fast trees.
- Paginas 46–48: cierre, referencias y acknowledgements.

Estructuras_de_Datos_Avanzadas_____Modelo_de_enteros_II.pdf

- Paginas 1–5: objetivos de RAM AC0 y Fusion Trees.
- Paginas 6–10: modelo RAM AC0, operaciones permitidas y ausencia de multiplicacion.
- Paginas 11–18: definicion de Fusion Trees y tiempos $O(\log_w n)$.
- Paginas 19–25: estructura tipo B-tree con nodos de fusion tree.
- Paginas 26–34: sketching usando trie binario y virtual tree.
- Paginas 35–40: sketching perfecto y preservacion de orden solo entre llaves.
- Paginas 41–48: consultas, sketch de query, LCP y punto de falla.
- Paginas 49–52: resumen y cierre.

Estructuras_de_Datos_Avanzadas_____Ordenamiento_en_tiempo_lineal.pdf

- Paginas 1–6: objetivos y relacion con priority queues.
- Paginas 7–11: equivalencia entre sorting y priority queues.
- Paginas 12–15: tabla de resultados de ordenamiento de enteros.
- Paginas 16–22: signature sort, condicion base y particion en digitos.
- Paginas 23–28: hashing perfecto de digitos y reduccion de bits.
- Paginas 29–34: packed sort y trie comprimido.
- Paginas 35–39: trie comprimido y problema de orden real.
- Paginas 40–46: ordenamiento recursivo de aristas del trie.
- Paginas 47–54: inorder traversal y resumen de signature sort.
- Paginas 55–59: resumen y cierre.

14 Apendice: mapa pagina por pagina de los slides de integer

Este apendice lista cada pagina/slide de la carpeta fuente y su contenido principal. Muchas paginas son animaciones repetidas de Beamer; se conservan para trazabilidad.

Estructuras_de_Datos_Avanzadas_____Modelo_de_enteros_II.pdf (52 paginas)

Pag. 1

CS3014 Estructuras de Datos Avanzadas CS3014 Estructuras de Datos Avanzadas Laboratorio - Semana 9 - Sesion 1 Victor Racso Galvan Oyola vgalvan@utec.edu.pe 20 de mayo de 2026 Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 1 / 14

Pag. 2

CS3014 Estructuras de Datos Avanzadas ?Que aprenderemos hoy? -Modelos computacional RAM AC 0 -Fusion Trees Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 2 / 14

Pag. 3

CS3014 Estructuras de Datos Avanzadas ?Que aprenderemos hoy? -Modelos computacional RAM AC 0 -Fusion Trees Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 2 / 14

Pag. 4

CS3014 Estructuras de Datos Avanzadas ?Que aprenderemos hoy? -Modelos computacional RAM AC 0 -Fusion Trees Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 2 / 14

Pag. 5

Modelos computacional RAM AC 0

Pag. 6

CS3014 Estructuras de Datos Avanzadas Modelos computacional RAM AC 0 Modelo RAM AC0 -Permite solo operaciones que se puedan realizar en un circuito de profundidad $O(1)$ pero sin limite de fan-in y fan-out (compuertas que entran y salen, respectivamente). -Por su definicion, no permite multiplicacion. Otras operaciones permitidas Operaciones bitwise pueden ser realizadas en $O(1)$. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 4 / 14

Pag. 7

CS3014 Estructuras de Datos Avanzadas Modelos computacional RAM AC 0 Modelo RAM AC0 -Permite solo operaciones que se puedan realizar en un circuito de profundidad $O(1)$ pero sin limite de fan-in y fan-out (compuertas que entran y salen, respectivamente). -Por su definicion, no permite multiplicacion. Otras operaciones permitidas Operaciones bitwise pueden ser realizadas en $O(1)$. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 4 / 14

Pag. 8

CS3014 Estructuras de Datos Avanzadas Modelos computacional RAM AC 0 Modelo RAM AC0 -Permite solo operaciones que se puedan realizar en un circuito de profundidad $O(1)$ pero sin limite de fan-in y fan-out (compuertas que entran y salen, respectivamente). -Por su definicion, no permite multiplicacion. Otras operaciones permitidas Operaciones bitwise pueden ser realizadas en $O(1)$. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 4 / 14

Pag. 9

CS3014 Estructuras de Datos Avanzadas Modelos computacional RAM AC 0 Modelo RAM AC0 -Permite solo operaciones que se puedan realizar en un circuito de profundidad $O(1)$ pero sin limite de fan-in y fan-out (compuertas que entran y salen, respectivamente). -Por su definicion, no permite multiplicacion. Otras operaciones permitidas Operaciones bitwise pueden ser realizadas en $O(1)$. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 4 / 14

Pag. 10

CS3014 Estructuras de Datos Avanzadas Modelos computacional RAM AC 0 Modelo RAM AC0 -Permite solo operaciones que se puedan realizar en un circuito de profundidad $O(1)$ pero sin limite de fan-in y fan-out (compuertas que entran y salen, respectivamente). -Por su definicion, no permite multiplicacion. Otras operaciones permitidas Operaciones bitwise pueden ser realizadas en $O(1)$. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 4 / 14

Pag. 11

Fusion Trees

Pag. 12

CS3014 Estructuras de Datos Avanzadas Fusion Trees Definicion Es una estructura de datos que nos permite consultar el sucesor/predecesor en $O(\log w \log n)$. Considerando este resultado y el de vEB y y-fast trees, tendríamos que la consulta de sucesor/predecesor toma $m' \in \{O(\log w), O(\log w \log n)\} = O(p \log n)$ Versiones Veremos la version estatica. Las dinamicas disponibles cumplen: $-O(\log w \log n + \log \log n)$ usando exponential trees. $-O(\log w \log n)$ esperado usando hashing. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 6 / 14

Pag. 13

CS3014 Estructuras de Datos Avanzadas Fusion Trees Definicion Es una estructura de datos que nos permite consultar el sucesor/predecesor en $O(\log w \log n)$. Considerando este resultado y el de vEB y y-fast trees, tendríamos que la consulta de sucesor/predecesor toma $m' \in \{O(\log w), O(\log w \log n)\} = O(p \log n)$ Versiones Veremos la version estatica. Las dinamicas disponibles cumplen: $-O(\log w \log n + \log \log n)$ usando exponential trees. $-O(\log w \log n)$ esperado usando hashing. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 6 / 14

Pag. 14

CS3014 Estructuras de Datos Avanzadas Fusion Trees Definicion Es una estructura de datos que nos permite consultar el sucesor/predecesor en $O(\log w \log n)$. Considerando este resultado y el de vEB y y-fast trees, tendríamos que la consulta de sucesor/predecesor toma $m' \in \{O(\log w), O(\log w \log n)\} = O(p \log n)$ Versiones Veremos la version estatica. Las dinamicas disponibles cumplen: $-O(\log w \log n + \log \log n)$ usando exponential trees. $-O(\log w \log n)$ esperado usando hashing. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 6 / 14

Pag. 15

CS3014 Estructuras de Datos Avanzadas Fusion Trees Definicion Es una estructura de datos que nos permite consultar el sucesor/predecesor en $O(\log w \log n)$. Considerando este resultado y el de vEB y y-fast trees, tendríamos que la consulta de sucesor/predecesor toma $m' \in \{O(\log w), O(\log w \log n)\} = O(p \log n)$ Versiones Veremos la version estatica. Las dinamicas disponibles cumplen: $-O(\log w \log n + \log \log n)$ usando exponential trees. $-O(\log w \log n)$ esperado usando hashing. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 6 / 14

Pag. 16

CS3014 Estructuras de Datos Avanzadas Fusion Trees Definicion Es una estructura de datos que nos permite consultar el sucesor/predecesor en $O(\log w \log n)$. Considerando este resultado y el de vEB y y-fast trees, tendríamos que la consulta de sucesor/predecesor toma $m' \in \{O(\log w), O(\log w \log n)\} = O(p \log n)$ Versiones Veremos la version estatica. Las dinamicas disponibles cumplen: $-O(\log w \log n + \log \log n)$ usando exponential trees. $-O(\log w \log n)$ esperado usando hashing. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 6 / 14

Pag. 17

CS3014 Estructuras de Datos Avanzadas Fusion Trees Definicion Es una estructura de datos que nos permite consultar el sucesor/predecesor en $O(\log w \log n)$. Considerando este resultado y el de vEB y y-fast trees, tendríamos que la consulta de sucesor/predecesor toma $m' \in \{O(\log w), O(\log w \log n)\} = O(p \log n)$ Versiones Veremos la version estatica. Las dinamicas disponibles cumplen: $-O(\log w \log n + \log \log n)$ usando exponential trees. $-O(\log w \log n)$ esperado usando hashing. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 6 / 14

Pag. 18

CS3014 Estructuras de Datos Avanzadas Fusion Trees Definicion Es una estructura de datos que nos permite consultar el sucesor/predecesor en $O(\log w \log n)$. Considerando este resultado y el de vEB y y-fast trees, tendríamos que la consulta de sucesor/predecesor toma $m' \in \{O(\log w), O(\log w \log n)\} = O(p \log n)$ Versiones Veremos la version estatica. Las dinamicas disponibles cumplen: $-O(\log w \log n + \log \log n)$ usando exponential trees. $-O(\log w \log n)$ esperado usando hashing. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 6 / 14

Pag. 19

CS3014 Estructuras de Datos Avanzadas Estructura Planteamiento inicial Consideraremos un B-Tree de orden $w^{1/5}$, así que cada nodo tiene a lo mucho $w^{1/5}$ hijos. Cada nodo maneja $w^{1/5} \log w$ bits, así que no podemos buscar el hijo correspondiente en $O(1)$. Solucion - Nodos de fusion tree - Almacenamos $k = O(w^{1/5})$ valores $x_0 < x_1 < \dots < x_{k-1}$. - Debemos poder responder al sucesor o predecesor de un valor en $O(1)$. - Requeriremos $O(1)$ de preprocesamiento. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 7 / 14

Pag. 20

CS3014 Estructuras de Datos Avanzadas Estructura Planteamiento inicial Consideraremos un B-Tree de orden w , así que cada nodo tiene a lo mucho w hijos. Cada nodo maneja $w \log w$ bits, así que no podemos buscar el hijo correspondiente en $O(1)$. Solución - Nodos de fusión tree - Almacenamos $\lceil \log w \rceil$ valores $x_0 < x_1 < \dots < x_{k-1}$. - Debemos poder responder al sucesor o predecesor de un valor en $O(1)$. - Requeriremos $O(1)$ de preprocesamiento. Victor Racso Galvan Oyola Universidad de Ingeniería y Tecnología 7 / 14

Pag. 21

CS3014 Estructuras de Datos Avanzadas Estructura Planteamiento inicial Consideraremos un B-Tree de orden w , así que cada nodo tiene a lo mucho w hijos. Cada nodo maneja $w \log w$ bits, así que no podemos buscar el hijo correspondiente en $O(1)$. Solución - Nodos de fusión tree - Almacenamos $\lceil \log w \rceil$ valores $x_0 < x_1 < \dots < x_{k-1}$. - Debemos poder responder al sucesor o predecesor de un valor en $O(1)$. - Requeriremos $O(1)$ de preprocesamiento. Victor Racso Galvan Oyola Universidad de Ingeniería y Tecnología 7 / 14

Pag. 22

CS3014 Estructuras de Datos Avanzadas Estructura Planteamiento inicial Consideraremos un B-Tree de orden w , así que cada nodo tiene a lo mucho w hijos. Cada nodo maneja $w \log w$ bits, así que no podemos buscar el hijo correspondiente en $O(1)$. Solución - Nodos de fusión tree - Almacenamos $\lceil \log w \rceil$ valores $x_0 < x_1 < \dots < x_{k-1}$. - Debemos poder responder al sucesor o predecesor de un valor en $O(1)$. - Requeriremos $O(1)$ de preprocesamiento. Victor Racso Galvan Oyola Universidad de Ingeniería y Tecnología 7 / 14

Pag. 23

CS3014 Estructuras de Datos Avanzadas Estructura Planteamiento inicial Consideraremos un B-Tree de orden w , así que cada nodo tiene a lo mucho w hijos. Cada nodo maneja $w \log w$ bits, así que no podemos buscar el hijo correspondiente en $O(1)$. Solución - Nodos de fusión tree - Almacenamos $\lceil \log w \rceil$ valores $x_0 < x_1 < \dots < x_{k-1}$. - Debemos poder responder al sucesor o predecesor de un valor en $O(1)$. - Requeriremos $O(1)$ de preprocesamiento. Victor Racso Galvan Oyola Universidad de Ingeniería y Tecnología 7 / 14

Pag. 24

CS3014 Estructuras de Datos Avanzadas Estructura Planteamiento inicial Consideraremos un B-Tree de orden w , así que cada nodo tiene a lo mucho w hijos. Cada nodo maneja $w \log w$ bits, así que no podemos buscar el hijo correspondiente en $O(1)$. Solución - Nodos de fusión tree - Almacenamos $\lceil \log w \rceil$ valores $x_0 < x_1 < \dots < x_{k-1}$. - Debemos poder responder al sucesor o predecesor de un valor en $O(1)$. - Requeriremos $O(1)$ de preprocesamiento. Victor Racso Galvan Oyola Universidad de Ingeniería y Tecnología 7 / 14

Pag. 25

CS3014 Estructuras de Datos Avanzadas Estructura Planteamiento inicial Consideraremos un B-Tree de orden w , así que cada nodo tiene a lo mucho w hijos. Cada nodo maneja $w \log w$ bits, así que no podemos buscar el hijo correspondiente en $O(1)$. Solución - Nodos de fusión tree - Almacenamos $\lceil \log w \rceil$ valores $x_0 < x_1 < \dots < x_{k-1}$. - Debemos poder responder al sucesor o predecesor de un valor en $O(1)$. - Requeriremos $O(1)$ de preprocesamiento. Victor Racso Galvan Oyola Universidad de Ingeniería y Tecnología 7 / 14

Pag. 26

CS3014 Estructuras de Datos Avanzadas Sketching ¿Cómo distinguimos $\lceil \log w \rceil$ valores? Podemos considerar un trie de las llaves con un alfabeto binario. Es posible reducir la cantidad total de nodos calculando el auxiliary/virtual tree del trie correspondiente y mapeamos los niveles involucrados. Consecuencias Hay $k-1$ nodos internos en el trie, así que nos interesan a lo mucho $k-1$ niveles del mismo. Si almacenamos la información de $k-1$ niveles, entonces manejaremos $O(w \log w)$ bits por cada llave. En total, manejaremos $O(w \log w)$ bits, lo cual si es posible...

Pag. 27

CS3014 Estructuras de Datos Avanzadas Sketching ¿Cómo distinguimos $\lceil \log w \rceil$ valores? Podemos considerar un trie de las llaves con un alfabeto binario. Es posible reducir la cantidad total de nodos calculando el auxiliary/virtual tree del trie correspondiente y mapeamos los niveles involucrados. Consecuencias Hay $k-1$ nodos internos en el trie, así que nos interesan a lo mucho $k-1$ niveles del mismo. Si almacenamos la información de $k-1$ niveles, entonces manejaremos $O(w \log w)$ bits por cada llave. En total, manejaremos $O(w \log w)$ bits, lo cual si es posible...

Pag. 28

CS3014 Estructuras de Datos Avanzadas Sketching ?Como distinguimos $k=O(w^{1/5})$ valores? Podemos considerar un trie de las llaves con un alfabeto binario. Es posible reducir la cantidad total de nodos calculando el auxiliary/virtual tree del trie correspondiente y mapeamos los niveles involucrados. Consecuencias Hayk-1 nodos internos en el trie, así que nos interesan a lo mucho $k-1$ niveles del mismo. Si almacenamos la información de $k-1$ niveles, entonces manejaremos $O(w^{1/5})$ bits por cada llave. En total, manejaremos $O(w^{2/5})$ bits, lo cual si es posible d...

Pag. 29

CS3014 Estructuras de Datos Avanzadas Sketching ?Como distinguimos $k=O(w^{1/5})$ valores? Podemos considerar un trie de las llaves con un alfabeto binario. Es posible reducir la cantidad total de nodos calculando el auxiliary/virtual tree del trie correspondiente y mapeamos los niveles involucrados. Consecuencias Hayk-1 nodos internos en el trie, así que nos interesan a lo mucho $k-1$ niveles del mismo. Si almacenamos la información de $k-1$ niveles, entonces manejaremos $O(w^{1/5})$ bits por cada llave. En total, manejaremos $O(w^{2/5})$ bits, lo cual si es posible d...

Pag. 30

CS3014 Estructuras de Datos Avanzadas Sketching ?Como distinguimos $k=O(w^{1/5})$ valores? Podemos considerar un trie de las llaves con un alfabeto binario. Es posible reducir la cantidad total de nodos calculando el auxiliary/virtual tree del trie correspondiente y mapeamos los niveles involucrados. Consecuencias Hayk-1 nodos internos en el trie, así que nos interesan a lo mucho $k-1$ niveles del mismo. Si almacenamos la información de $k-1$ niveles, entonces manejaremos $O(w^{1/5})$ bits por cada llave. En total, manejaremos $O(w^{2/5})$ bits, lo cual si es posible d...

Pag. 31

CS3014 Estructuras de Datos Avanzadas Sketching ?Como distinguimos $k=O(w^{1/5})$ valores? Podemos considerar un trie de las llaves con un alfabeto binario. Es posible reducir la cantidad total de nodos calculando el auxiliary/virtual tree del trie correspondiente y mapeamos los niveles involucrados. Consecuencias Hayk-1 nodos internos en el trie, así que nos interesan a lo mucho $k-1$ niveles del mismo. Si almacenamos la información de $k-1$ niveles, entonces manejaremos $O(w^{1/5})$ bits por cada llave. En total, manejaremos $O(w^{2/5})$ bits, lo cual si es posible d...

Pag. 32

CS3014 Estructuras de Datos Avanzadas Sketching ?Como distinguimos $k=O(w^{1/5})$ valores? Podemos considerar un trie de las llaves con un alfabeto binario. Es posible reducir la cantidad total de nodos calculando el auxiliary/virtual tree del trie correspondiente y mapeamos los niveles involucrados. Consecuencias Hayk-1 nodos internos en el trie, así que nos interesan a lo mucho $k-1$ niveles del mismo. Si almacenamos la información de $k-1$ niveles, entonces manejaremos $O(w^{1/5})$ bits por cada llave. En total, manejaremos $O(w^{2/5})$ bits, lo cual si es posible d...

Pag. 33

CS3014 Estructuras de Datos Avanzadas Trie binario empty 1 1 0 2 0 3 1 1 0 5 1 0 1 1=001,2=010,3=011,5=101 Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 9 / 14

Pag. 34

CS3014 Estructuras de Datos Avanzadas Trie binario + Virtual Tree empty 1 1 0 2 0 3 1 1 0 5 1 0 1 1=001,2=010,3=011,5=101 Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 10 / 14

Pag. 35

CS3014 Estructuras de Datos Avanzadas Sketching perfecto Definicion Consideraremos una secuencia de enteros $b_0 > b_1 > \dots > b_{r-1}$ y cada llave la representaremos como una cadena de bits en la cual el i -ésimo bit corresponderá al bit b_i de la llave. Denotaremos a dicha cadena binaria como $\text{Sketch}(x)$. Propiedad importante $\text{Sketch}(x_0) < \text{Sketch}(x_1) < \dots < \text{Sketch}(x_{k-1})$ Esto se cumple solo para las llaves. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 11 / 14

Pag. 36

CS3014 Estructuras de Datos Avanzadas Sketching perfecto Definicion Consideraremos una secuencia de enteros $b_0 > b_1 > \dots > b_{r-1}$ y cada llave la representaremos como una cadena de bits en la cual el i -ésimo

bit correspondera al bit b_i de la llave. Denotaremos a dicha cadena binaria como $\text{Sketch}(x)$. Propiedad importante $\text{Sketch}(x_0) < \text{Sketch}(x_1) < \dots < \text{Sketch}(x_{k-1})$ Esto se cumple solo para las llaves. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 11 / 14

Pag. 37

CS3014 Estructuras de Datos Avanzadas Sketching perfecto Definicion Consideraremos una secuencia de enteros $b_0 > b_1 > \dots > b_{r-1}$ y cada llave la representaremos como una cadena de bits en la cual el i -esimo bit correspondera al bit b_i de la llave. Denotaremos a dicha cadena binaria como $\text{Sketch}(x)$. Propiedad importante $\text{Sketch}(x_0) < \text{Sketch}(x_1) < \dots < \text{Sketch}(x_{k-1})$ Esto se cumple solo para las llaves. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 11 / 14

Pag. 38

CS3014 Estructuras de Datos Avanzadas Sketching perfecto Definicion Consideraremos una secuencia de enteros $b_0 > b_1 > \dots > b_{r-1}$ y cada llave la representaremos como una cadena de bits en la cual el i -esimo bit correspondera al bit b_i de la llave. Denotaremos a dicha cadena binaria como $\text{Sketch}(x)$. Propiedad importante $\text{Sketch}(x_0) < \text{Sketch}(x_1) < \dots < \text{Sketch}(x_{k-1})$ Esto se cumple solo para las llaves. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 11 / 14

Pag. 39

CS3014 Estructuras de Datos Avanzadas Sketching perfecto Definicion Consideraremos una secuencia de enteros $b_0 > b_1 > \dots > b_{r-1}$ y cada llave la representaremos como una cadena de bits en la cual el i -esimo bit correspondera al bit b_i de la llave. Denotaremos a dicha cadena binaria como $\text{Sketch}(x)$. Propiedad importante $\text{Sketch}(x_0) < \text{Sketch}(x_1) < \dots < \text{Sketch}(x_{k-1})$ Esto se cumple solo para las llaves. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 11 / 14

Pag. 40

CS3014 Estructuras de Datos Avanzadas Sketching perfecto Definicion Consideraremos una secuencia de enteros $b_0 > b_1 > \dots > b_{r-1}$ y cada llave la representaremos como una cadena de bits en la cual el i -esimo bit correspondera al bit b_i de la llave. Denotaremos a dicha cadena binaria como $\text{Sketch}(x)$. Propiedad importante $\text{Sketch}(x_0) < \text{Sketch}(x_1) < \dots < \text{Sketch}(x_{k-1})$ Esto se cumple solo para las llaves. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 11 / 14

Pag. 41

CS3014 Estructuras de Datos Avanzadas Consultas ?Como buscamos el sucesor o predecesor de un valor? Podemos pensar en calcular $\text{Sketch}(q)$ y luego buscarlo en la estructura. Sorpresa: Sketch preserva el orden entre las llaves, así que $\text{Sketch}(q)$ podría caer en cualquier lugar, no necesariamente en su sucesor o predecesor. ?Que hacemos? Supongamos que $\text{Sketch}(x_i) \leq \text{Sketch}(q) < \text{Sketch}(x_{i+1})$. Tomamos el longest common prefix entre q y x_{i+1} . El nodo al que lleguemos es en donde la búsqueda falla, así que podremos decidir que hacer a continuación dependiendo del escenario...

Pag. 42

CS3014 Estructuras de Datos Avanzadas Consultas ?Como buscamos el sucesor o predecesor de un valor? Podemos pensar en calcular $\text{Sketch}(q)$ y luego buscarlo en la estructura. Sorpresa: Sketch preserva el orden entre las llaves, así que $\text{Sketch}(q)$ podría caer en cualquier lugar, no necesariamente en su sucesor o predecesor. ?Que hacemos? Supongamos que $\text{Sketch}(x_i) \leq \text{Sketch}(q) < \text{Sketch}(x_{i+1})$. Tomamos el longest common prefix entre q y x_{i+1} . El nodo al que lleguemos es en donde la búsqueda falla, así que podremos decidir que hacer a continuación dependiendo del escenario...

Pag. 43

CS3014 Estructuras de Datos Avanzadas Consultas ?Como buscamos el sucesor o predecesor de un valor? Podemos pensar en calcular $\text{Sketch}(q)$ y luego buscarlo en la estructura. Sorpresa: Sketch preserva el orden entre las llaves, así que $\text{Sketch}(q)$ podría caer en cualquier lugar, no necesariamente en su sucesor o predecesor. ?Que hacemos? Supongamos que $\text{Sketch}(x_i) \leq \text{Sketch}(q) < \text{Sketch}(x_{i+1})$. Tomamos el longest common prefix entre q y x_{i+1} . El nodo al que lleguemos es en donde la búsqueda falla, así que podremos decidir que hacer a continuación dependiendo del escenario...

Pag. 44

CS3014 Estructuras de Datos Avanzadas Consultas ?Como buscamos el sucesor o predecesor de un valor? Podemos pensar en calcular $\text{Sketch}(q)$ y luego buscarlo en la estructura. Sorpresa: Sketch preserva el orden entre las llaves, así que $\text{Sketch}(q)$ podría caer en cualquier lugar, no necesariamente en su sucesor o predecesor. ?Que

hacemos? Supongamos que $\text{Sketch}(x_i) \leq \text{Sketch}(q) < \text{Sketch}(x_{i+1})$. Tomamos el longest common prefix entre q y x_{i+1} . El nodo al que lleguemos es en donde la búsqueda falla, así que podremos decidir que hacer a continuación dependiendo del escenario...

Pag. 45

CS3014 Estructuras de Datos Avanzadas Consultas ¿Cómo buscamos el sucesor o predecesor de un valor q ? Podemos pensar en calcular $\text{Sketch}(q)$ y luego buscarlo en la estructura. Sorpresa: Sketch preserva el orden entre las llaves, así que $\text{Sketch}(q)$ podría caer en cualquier lugar, no necesariamente en su sucesor o predecesor. ¿Qué hacemos? Supongamos que $\text{Sketch}(x_i) \leq \text{Sketch}(q) < \text{Sketch}(x_{i+1})$. Tomamos el longest common prefix entre q y x_{i+1} . El nodo al que lleguemos es en donde la búsqueda falla, así que podremos decidir que hacer a continuación dependiendo del escenario...

Pag. 46

CS3014 Estructuras de Datos Avanzadas Consultas ¿Cómo buscamos el sucesor o predecesor de un valor q ? Podemos pensar en calcular $\text{Sketch}(q)$ y luego buscarlo en la estructura. Sorpresa: Sketch preserva el orden entre las llaves, así que $\text{Sketch}(q)$ podría caer en cualquier lugar, no necesariamente en su sucesor o predecesor. ¿Qué hacemos? Supongamos que $\text{Sketch}(x_i) \leq \text{Sketch}(q) < \text{Sketch}(x_{i+1})$. Tomamos el longest common prefix entre q y x_{i+1} . El nodo al que lleguemos es en donde la búsqueda falla, así que podremos decidir que hacer a continuación dependiendo del escenario...

Pag. 47

CS3014 Estructuras de Datos Avanzadas Consultas ¿Cómo buscamos el sucesor o predecesor de un valor q ? Podemos pensar en calcular $\text{Sketch}(q)$ y luego buscarlo en la estructura. Sorpresa: Sketch preserva el orden entre las llaves, así que $\text{Sketch}(q)$ podría caer en cualquier lugar, no necesariamente en su sucesor o predecesor. ¿Qué hacemos? Supongamos que $\text{Sketch}(x_i) \leq \text{Sketch}(q) < \text{Sketch}(x_{i+1})$. Tomamos el longest common prefix entre q y x_{i+1} . El nodo al que lleguemos es en donde la búsqueda falla, así que podremos decidir que hacer a continuación dependiendo del escenario...

Pag. 48

CS3014 Estructuras de Datos Avanzadas Consultas ¿Cómo buscamos el sucesor o predecesor de un valor q ? Podemos pensar en calcular $\text{Sketch}(q)$ y luego buscarlo en la estructura. Sorpresa: Sketch preserva el orden entre las llaves, así que $\text{Sketch}(q)$ podría caer en cualquier lugar, no necesariamente en su sucesor o predecesor. ¿Qué hacemos? Supongamos que $\text{Sketch}(x_i) \leq \text{Sketch}(q) < \text{Sketch}(x_{i+1})$. Tomamos el longest common prefix entre q y x_{i+1} . El nodo al que lleguemos es en donde la búsqueda falla, así que podremos decidir que hacer a continuación dependiendo del escenario...

Pag. 49

CS3014 Estructuras de Datos Avanzadas Resumen de la sesión -Aprendimos un poco del modelo RAM AC0 .
-Aprendimos sobre Fusion Trees. Victor Racso Galvan Oyola Universidad de Ingeniería y Tecnología 13 / 14

Pag. 50

CS3014 Estructuras de Datos Avanzadas Resumen de la sesión -Aprendimos un poco del modelo RAM AC0 .
-Aprendimos sobre Fusion Trees. Victor Racso Galvan Oyola Universidad de Ingeniería y Tecnología 13 / 14

Pag. 51

CS3014 Estructuras de Datos Avanzadas Resumen de la sesión -Aprendimos un poco del modelo RAM AC0 .
-Aprendimos sobre Fusion Trees. Victor Racso Galvan Oyola Universidad de Ingeniería y Tecnología 13 / 14

Pag. 52

CS3014 Estructuras de Datos Avanzadas Gracias Victor Racso Galvan Oyola Universidad de Ingeniería y Tecnología 14 / 14

Estructuras de Datos Avanzadas Ordenamiento en tiempo lineal.pdf (59 páginas)

Pag. 1

CS3014 Estructuras de Datos Avanzadas CS3014 Estructuras de Datos Avanzadas Laboratorio - Semana 9 - Sesión 2 Victor Racso Galvan Oyola vgalvan@utec.edu.pe 22 de mayo de 2026 Victor Racso Galvan Oyola Universidad de Ingeniería y Tecnología 1 / 15

Pag. 2

CS3014 Estructuras de Datos Avanzadas ?Que aprenderemos hoy? -Equivalencia entre ordenamiento y colas de prioridad -Algunos resultados en ordenamiento de enteros -Signature sort Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 2 / 15

Pag. 3

CS3014 Estructuras de Datos Avanzadas ?Que aprenderemos hoy? -Equivalencia entre ordenamiento y colas de prioridad -Algunos resultados en ordenamiento de enteros -Signature sort Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 2 / 15

Pag. 4

CS3014 Estructuras de Datos Avanzadas ?Que aprenderemos hoy? -Equivalencia entre ordenamiento y colas de prioridad -Algunos resultados en ordenamiento de enteros -Signature sort Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 2 / 15

Pag. 5

CS3014 Estructuras de Datos Avanzadas ?Que aprenderemos hoy? -Equivalencia entre ordenamiento y colas de prioridad -Algunos resultados en ordenamiento de enteros -Signature sort Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 2 / 15

Pag. 6

Ordenamiento y cola de prioridades

Pag. 7

CS3014 Estructuras de Datos Avanzadas Equivalencia Punto inicial Es sencillo notar que si tenemos una cola de prioridad que soporta las operaciones clásicas en $O(f(n,w))$, entonces existe un algoritmo de ordenamiento que toma $O(nf(n,w))$. Punto final Si existe un algoritmo de ordenamiento que toma $O(T(n,w))$, existe una cola de prioridad que soporta sus operaciones clásicas en $O(T(n,w) \cdot n)$. Ejemplo: Quickheap. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 4 / 15

Pag. 8

CS3014 Estructuras de Datos Avanzadas Equivalencia Punto inicial Es sencillo notar que si tenemos una cola de prioridad que soporta las operaciones clásicas en $O(f(n,w))$, entonces existe un algoritmo de ordenamiento que toma $O(nf(n,w))$. Punto final Si existe un algoritmo de ordenamiento que toma $O(T(n,w))$, existe una cola de prioridad que soporta sus operaciones clásicas en $O(T(n,w) \cdot n)$. Ejemplo: Quickheap. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 4 / 15

Pag. 9

CS3014 Estructuras de Datos Avanzadas Equivalencia Punto inicial Es sencillo notar que si tenemos una cola de prioridad que soporta las operaciones clásicas en $O(f(n,w))$, entonces existe un algoritmo de ordenamiento que toma $O(nf(n,w))$. Punto final Si existe un algoritmo de ordenamiento que toma $O(T(n,w))$, existe una cola de prioridad que soporta sus operaciones clásicas en $O(T(n,w) \cdot n)$. Ejemplo: Quickheap. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 4 / 15

Pag. 10

CS3014 Estructuras de Datos Avanzadas Equivalencia Punto inicial Es sencillo notar que si tenemos una cola de prioridad que soporta las operaciones clásicas en $O(f(n,w))$, entonces existe un algoritmo de ordenamiento que toma $O(nf(n,w))$. Punto final Si existe un algoritmo de ordenamiento que toma $O(T(n,w))$, existe una cola de prioridad que soporta sus operaciones clásicas en $O(T(n,w) \cdot n)$. Ejemplo: Quickheap. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 4 / 15

Pag. 11

CS3014 Estructuras de Datos Avanzadas Equivalencia Punto inicial Es sencillo notar que si tenemos una cola de prioridad que soporta las operaciones clásicas en $O(f(n,w))$, entonces existe un algoritmo de ordenamiento que toma $O(nf(n,w))$. Punto final Si existe un algoritmo de ordenamiento que toma $O(T(n,w))$, existe una cola de prioridad que soporta sus operaciones clásicas en $O(T(n,w) \cdot n)$. Ejemplo: Quickheap. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 4 / 15

Pag. 12

Resultados en ordenamientos de enteros

Pag. 13

CS3014 Estructuras de Datos Avanzadas Resultados en ordenamiento de enteros Para ordenar n enteros de w bits, la siguiente tabla resume algunos resultados historicos. Metodo Complejidad Condicion Comparaciones
 $O(n \log n)$ Sin condiciones Counting sort $O(n + 2^w)$ $O(n)$ si $w = \log n$ Radix sort $O(n w \log n)$ $O(n)$ si $w = O(\log n)$
van Emde Boas $O(n \log w)$ Basado en van Emde Boas $O(n \log w \log n)$ $O(n \log \log n)$ si $w = \log O(1) n$ Signature
sort $O(n w)$ $w = \Omega(\log^2 + \epsilon n)$, for all $\epsilon > 0$ Han (2004) $O(n \log \log n)$ modelo RAM AC0 Han y
Thorup (2002) $O(n q \log w \log n)$ Aleatorizado Cuadro...

Pag. 14

CS3014 Estructuras de Datos Avanzadas Resultados en ordenamiento de enteros Para ordenar n enteros de w bits, la siguiente tabla resume algunos resultados historicos. Metodo Complejidad Condicion Comparaciones
 $O(n \log n)$ Sin condiciones Counting sort $O(n + 2^w)$ $O(n)$ si $w = \log n$ Radix sort $O(n w \log n)$ $O(n)$ si $w = O(\log n)$
van Emde Boas $O(n \log w)$ Basado en van Emde Boas $O(n \log w \log n)$ $O(n \log \log n)$ si $w = \log O(1) n$ Signature
sort $O(n w)$ $w = \Omega(\log^2 + \epsilon n)$, for all $\epsilon > 0$ Han (2004) $O(n \log \log n)$ modelo RAM AC0 Han y
Thorup (2002) $O(n q \log w \log n)$ Aleatorizado Cuadro...

Pag. 15

CS3014 Estructuras de Datos Avanzadas Resultados en ordenamiento de enteros Para ordenar n enteros de w bits, la siguiente tabla resume algunos resultados historicos. Metodo Complejidad Condicion Comparaciones
 $O(n \log n)$ Sin condiciones Counting sort $O(n + 2^w)$ $O(n)$ si $w = \log n$ Radix sort $O(n w \log n)$ $O(n)$ si $w = O(\log n)$
van Emde Boas $O(n \log w)$ Basado en van Emde Boas $O(n \log w \log n)$ $O(n \log \log n)$ si $w = \log O(1) n$ Signature
sort $O(n w)$ $w = \Omega(\log^2 + \epsilon n)$, for all $\epsilon > 0$ Han (2004) $O(n \log \log n)$ modelo RAM AC0 Han y
Thorup (2002) $O(n q \log w \log n)$ Aleatorizado Cuadro...

Pag. 16

Signature sort

Pag. 17

CS3014 Estructuras de Datos Avanzadas Signature sort Condicion base Asumiremos que $w \geq \log^2 + \epsilon n$ por ser conveniente. Paso 1 Particionamos cada entero en $\log^2 + \epsilon n$ bloques de tamanos iguales que seran digitos. Notemos que estos bloques tendran tamano de al menos $\log^2 n$. Esto es equivalente a tomar una base $B \sim w \log^2 + \epsilon n$ y representar cada entero en base B . Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 8 / 15

Pag. 18

CS3014 Estructuras de Datos Avanzadas Signature sort Condicion base Asumiremos que $w \geq \log^2 + \epsilon n$ por ser conveniente. Paso 1 Particionamos cada entero en $\log^2 + \epsilon n$ bloques de tamanos iguales que seran digitos. Notemos que estos bloques tendran tamano de al menos $\log^2 n$. Esto es equivalente a tomar una base $B \sim w \log^2 + \epsilon n$ y representar cada entero en base B . Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 8 / 15

Pag. 19

CS3014 Estructuras de Datos Avanzadas Signature sort Condicion base Asumiremos que $w \geq \log^2 + \epsilon n$ por ser conveniente. Paso 1 Particionamos cada entero en $\log^2 + \epsilon n$ bloques de tamanos iguales que seran digitos. Notemos que estos bloques tendran tamano de al menos $\log^2 n$. Esto es equivalente a tomar una base $B \sim w \log^2 + \epsilon n$ y representar cada entero en base B . Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 8 / 15

Pag. 20

CS3014 Estructuras de Datos Avanzadas Signature sort Condicion base Asumiremos que $w \geq \log^2 + \epsilon n$ por ser conveniente. Paso 1 Particionamos cada entero en $\log^2 + \epsilon n$ bloques de tamanos iguales que seran digitos. Notemos que estos bloques tendran tamano de al menos $\log^2 n$. Esto es equivalente a tomar una base $B \sim w \log^2 + \epsilon n$ y representar cada entero en base B . Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 8 / 15

Pag. 21

CS3014 Estructuras de Datos Avanzadas Signature sort Condicion base Asumiremos que $w \geq \log^2 + \epsilon n$ por ser conveniente. Paso 1 Particionamos cada entero en $\log^2 + \epsilon n$ bloques de tamanos iguales que seran digitos. Notemos que estos bloques tendran tamano de al menos $\log^2 n$. Esto es equivalente a tomar

una base $B \sim w \log \epsilon n$ y representar cada entero en base B . Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 8 / 15

Pag. 22

CS3014 Estructuras de Datos Avanzadas Signature sort Condicion base Asumiremos que $w \geq \log 2 + \epsilon$ $n \log n$ por ser conveniente. Paso 1 Particionamos cada entero en $\log \epsilon n$ bloques de tamanos iguales que seran digitos. Notemos que estos bloques tendran tamano de al menos $\log 2 n$. Esto es equivalente a tomar una base $B \sim w \log \epsilon n$ y representar cada entero en base B . Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 8 / 15

Pag. 23

CS3014 Estructuras de Datos Avanzadas Signature sort Paso 2 Ya que hay pocos posibles digitos almacenados para un universo muy grande ($n \log 1 + \epsilon n \ll 2 \log 2 n$), reemplazamos cada bloque por un hash perfecto de $O(\log n)$ bits. Notemos que podriamos elegir una constante grande para evitar colisiones con alta probabilidad. Podemos multiplicar cada digito por un entero y luego obtener las mascaras reducidas de cada uno. Esto tomara $O(1)$ por digito. Esto preserva la identidad pero no preserva el orden. Paso 3 Notemos que ahora la cantidad de bits usados es $O(n \log n)$.

Pag. 24

CS3014 Estructuras de Datos Avanzadas Signature sort Paso 2 Ya que hay pocos posibles digitos almacenados para un universo muy grande ($n \log 1 + \epsilon n \ll 2 \log 2 n$), reemplazamos cada bloque por un hash perfecto de $O(\log n)$ bits. Notemos que podriamos elegir una constante grande para evitar colisiones con alta probabilidad. Podemos multiplicar cada digito por un entero y luego obtener las mascaras reducidas de cada uno. Esto tomara $O(1)$ por digito. Esto preserva la identidad pero no preserva el orden. Paso 3 Notemos que ahora la cantidad de bits usados es $O(n \log n)$.

Pag. 25

CS3014 Estructuras de Datos Avanzadas Signature sort Paso 2 Ya que hay pocos posibles digitos almacenados para un universo muy grande ($n \log 1 + \epsilon n \ll 2 \log 2 n$), reemplazamos cada bloque por un hash perfecto de $O(\log n)$ bits. Notemos que podriamos elegir una constante grande para evitar colisiones con alta probabilidad. Podemos multiplicar cada digito por un entero y luego obtener las mascaras reducidas de cada uno. Esto tomara $O(1)$ por digito. Esto preserva la identidad pero no preserva el orden. Paso 3 Notemos que ahora la cantidad de bits usados es $O(n \log n)$.

Pag. 26

CS3014 Estructuras de Datos Avanzadas Signature sort Paso 2 Ya que hay pocos posibles digitos almacenados para un universo muy grande ($n \log 1 + \epsilon n \ll 2 \log 2 n$), reemplazamos cada bloque por un hash perfecto de $O(\log n)$ bits. Notemos que podriamos elegir una constante grande para evitar colisiones con alta probabilidad. Podemos multiplicar cada digito por un entero y luego obtener las mascaras reducidas de cada uno. Esto tomara $O(1)$ por digito. Esto preserva la identidad pero no preserva el orden. Paso 3 Notemos que ahora la cantidad de bits usados es $O(n \log n)$.

Pag. 27

CS3014 Estructuras de Datos Avanzadas Signature sort Paso 2 Ya que hay pocos posibles digitos almacenados para un universo muy grande ($n \log 1 + \epsilon n \ll 2 \log 2 n$), reemplazamos cada bloque por un hash perfecto de $O(\log n)$ bits. Notemos que podriamos elegir una constante grande para evitar colisiones con alta probabilidad. Podemos multiplicar cada digito por un entero y luego obtener las mascaras reducidas de cada uno. Esto tomara $O(1)$ por digito. Esto preserva la identidad pero no preserva el orden. Paso 3 Notemos que ahora la cantidad de bits usados es $O(n \log n)$.

Pag. 28

CS3014 Estructuras de Datos Avanzadas Signature sort Paso 2 Ya que hay pocos posibles digitos almacenados para un universo muy grande ($n \log 1 + \epsilon n \ll 2 \log 2 n$), reemplazamos cada bloque por un hash perfecto de $O(\log n)$ bits. Notemos que podriamos elegir una constante grande para evitar colisiones con alta probabilidad. Podemos multiplicar cada digito por un entero y luego obtener las mascaras reducidas de cada uno. Esto tomara $O(1)$ por digito. Esto preserva la identidad pero no preserva el orden. Paso 3 Notemos que ahora la cantidad de bits usados es $O(n \log n)$.

Pag. 29

CS3014 Estructuras de Datos Avanzadas Signature sort Packed sort Es un algoritmo que nos permite ordenar enteros de $O(\log(1+\epsilon)n)$ bits, así que cumplimos la condición sin problemas sobre los hashes/signatures. Por lo tanto, ordenaremos los hashes en $O(n)$. Paso 4 Construimos un trie comprimido sobre los hashes, de manera que el inorder traversal del mismo nos da los hashes ordenados. Esto toma $O(n)$ de espacio y es posible construirlo en $O(n)$ también. Vict...

Pag. 30

CS3014 Estructuras de Datos Avanzadas Signature sort Packed sort Es un algoritmo que nos permite ordenar enteros de $O(\log(1+\epsilon)n)$ bits, así que cumplimos la condición sin problemas sobre los hashes/signatures. Por lo tanto, ordenaremos los hashes en $O(n)$. Paso 4 Construimos un trie comprimido sobre los hashes, de manera que el inorder traversal del mismo nos da los hashes ordenados. Esto toma $O(n)$ de espacio y es posible construirlo en $O(n)$ también. Vict...

Pag. 31

CS3014 Estructuras de Datos Avanzadas Signature sort Packed sort Es un algoritmo que nos permite ordenar enteros de $O(\log(1+\epsilon)n)$ bits, así que cumplimos la condición sin problemas sobre los hashes/signatures. Por lo tanto, ordenaremos los hashes en $O(n)$. Paso 4 Construimos un trie comprimido sobre los hashes, de manera que el inorder traversal del mismo nos da los hashes ordenados. Esto toma $O(n)$ de espacio y es posible construirlo en $O(n)$ también. Vict...

Pag. 32

CS3014 Estructuras de Datos Avanzadas Signature sort Packed sort Es un algoritmo que nos permite ordenar enteros de $O(\log(1+\epsilon)n)$ bits, así que cumplimos la condición sin problemas sobre los hashes/signatures. Por lo tanto, ordenaremos los hashes en $O(n)$. Paso 4 Construimos un trie comprimido sobre los hashes, de manera que el inorder traversal del mismo nos da los hashes ordenados. Esto toma $O(n)$ de espacio y es posible construirlo en $O(n)$ también. Vict...

Pag. 33

CS3014 Estructuras de Datos Avanzadas Signature sort Packed sort Es un algoritmo que nos permite ordenar enteros de $O(\log(1+\epsilon)n)$ bits, así que cumplimos la condición sin problemas sobre los hashes/signatures. Por lo tanto, ordenaremos los hashes en $O(n)$. Paso 4 Construimos un trie comprimido sobre los hashes, de manera que el inorder traversal del mismo nos da los hashes ordenados. Esto toma $O(n)$ de espacio y es posible construirlo en $O(n)$ también. Vict...

Pag. 34

CS3014 Estructuras de Datos Avanzadas Signature sort Packed sort Es un algoritmo que nos permite ordenar enteros de $O(\log(1+\epsilon)n)$ bits, así que cumplimos la condición sin problemas sobre los hashes/signatures. Por lo tanto, ordenaremos los hashes en $O(n)$. Paso 4 Construimos un trie comprimido sobre los hashes, de manera que el inorder traversal del mismo nos da los hashes ordenados. Esto toma $O(n)$ de espacio y es posible construirlo en $O(n)$ también. Vict...

Pag. 35

CS3014 Estructuras de Datos Avanzadas Signature sort Trie comprimido Tomaremos cada nodo del trie con alfabeto $\{0, 1, \dots, B-1\}$ que no tiene al menos dos hijos almacenados y lo "uniremos" con su padre. Debido a lo anterior, tendremos $O(n)$ nodos en la versión comprimida, comparado con los $O(n \log \epsilon n)$ que tendríamos en la versión completa. Problemática Tenemos todos los enteros originales ordenados según sus hashes, así que en realidad no tenemos ningún orden correcto todavía. Victor Racso Galvan Oyola Universidad de Ingeniería y Tecnología 11 / 15

Pag. 36

CS3014 Estructuras de Datos Avanzadas Signature sort Trie comprimido Tomaremos cada nodo del trie con alfabeto $\{0, 1, \dots, B-1\}$ que no tiene al menos dos hijos almacenados y lo "uniremos" con su padre. Debido a lo

anterior, tendremos $O(n)$ nodos en la version comprimida, comparado con los $O(n \log \epsilon n)$ que tendríamos en la version completa. Problemática Tenemos todos los enteros originales ordenados segun sus hashes, así que en realidad no tenemos ningun orden correcto todavia. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 11 / 15

Pag. 37

CS3014 Estructuras de Datos Avanzadas Signature sort Trie comprimido Tomaremos cada nodo del trie con alfabeto $\{0,1, \dots, B-1\}$ que no tiene al menos dos hijos almacenados y lo "uniremos" con su padre. Debido a lo anterior, tendremos $O(n)$ nodos en la version comprimida, comparado con los $O(n \log \epsilon n)$ que tendríamos en la version completa. Problemática Tenemos todos los enteros originales ordenados segun sus hashes, así que en realidad no tenemos ningun orden correcto todavia. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 11 / 15

Pag. 38

CS3014 Estructuras de Datos Avanzadas Signature sort Trie comprimido Tomaremos cada nodo del trie con alfabeto $\{0,1, \dots, B-1\}$ que no tiene al menos dos hijos almacenados y lo "uniremos" con su padre. Debido a lo anterior, tendremos $O(n)$ nodos en la version comprimida, comparado con los $O(n \log \epsilon n)$ que tendríamos en la version completa. Problemática Tenemos todos los enteros originales ordenados segun sus hashes, así que en realidad no tenemos ningun orden correcto todavia. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 11 / 15

Pag. 39

CS3014 Estructuras de Datos Avanzadas Signature sort Trie comprimido Tomaremos cada nodo del trie con alfabeto $\{0,1, \dots, B-1\}$ que no tiene al menos dos hijos almacenados y lo "uniremos" con su padre. Debido a lo anterior, tendremos $O(n)$ nodos en la version comprimida, comparado con los $O(n \log \epsilon n)$ que tendríamos en la version completa. Problemática Tenemos todos los enteros originales ordenados segun sus hashes, así que en realidad no tenemos ningun orden correcto todavia. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 11 / 15

Pag. 40

CS3014 Estructuras de Datos Avanzadas Signature sort Paso 5 Ordenaremos las aristas del trie recursivamente usando (Indice del nodo, Primer digito, Indice del hijo). De esta manera, si el nivel actual tiene enteros de w bits, estamos reduciendolos a una tupla de $(O(\log n), O(w \log \epsilon n), O(\log n))$, así que la cantidad de bits se reduce a $w \log \epsilon n + O(\log n)$. Luego de 1 epsilon +1 niveles de recursion se tendra que la cantidad de bits de cada entero es $b = O(1 \epsilon \log n + w \log(1 + \epsilon n)) = O(w \log \log \log n)$. Sobre este escenario podemos usar...

Pag. 41

CS3014 Estructuras de Datos Avanzadas Signature sort Paso 5 Ordenaremos las aristas del trie recursivamente usando (Indice del nodo, Primer digito, Indice del hijo). De esta manera, si el nivel actual tiene enteros de w bits, estamos reduciendolos a una tupla de $(O(\log n), O(w \log \epsilon n), O(\log n))$, así que la cantidad de bits se reduce a $w \log \epsilon n + O(\log n)$. Luego de 1 epsilon +1 niveles de recursion se tendra que la cantidad de bits de cada entero es $b = O(1 \epsilon \log n + w \log(1 + \epsilon n)) = O(w \log \log \log n)$. Sobre este escenario podemos usar...

Pag. 42

CS3014 Estructuras de Datos Avanzadas Signature sort Paso 5 Ordenaremos las aristas del trie recursivamente usando (Indice del nodo, Primer digito, Indice del hijo). De esta manera, si el nivel actual tiene enteros de w bits, estamos reduciendolos a una tupla de $(O(\log n), O(w \log \epsilon n), O(\log n))$, así que la cantidad de bits se reduce a $w \log \epsilon n + O(\log n)$. Luego de 1 epsilon +1 niveles de recursion se tendra que la cantidad de bits de cada entero es $b = O(1 \epsilon \log n + w \log(1 + \epsilon n)) = O(w \log \log \log n)$. Sobre este escenario podemos usar...

Pag. 43

CS3014 Estructuras de Datos Avanzadas Signature sort Paso 5 Ordenaremos las aristas del trie recursivamente usando (Indice del nodo, Primer digito, Indice del hijo). De esta manera, si el nivel actual tiene enteros de w bits, estamos reduciendolos a una tupla de $(O(\log n), O(w \log \epsilon n), O(\log n))$, así que la cantidad de bits se reduce a $w \log \epsilon n + O(\log n)$. Luego de 1 epsilon +1 niveles de recursion se tendra que la cantidad de bits de cada entero es $b = O(1 \epsilon \log n + w \log(1 + \epsilon n)) = O(w \log \log \log n)$. Sobre este escenario podemos usar...

Pag. 44

CS3014 Estructuras de Datos Avanzadas Signature sort Paso 5 Ordenaremos las aristas del trie recursivamente usando (Indice del nodo, Primer digito, Indice del hijo). De esta manera, si el nivel actual tiene enteros dew' bits, estamos reduciendo a una tupla de $(O(\log n), O(w' \log \epsilon n), O(\log n))$, así que la cantidad de bits se reduce a $w' \log \epsilon n + O(\log n)$. Luego de 1 epsilon +1 niveles de recursion se tendra que la cantidad de bits de cada entero es $b = O(1 \epsilon \log n + w' \log(1 + \epsilon n)) = O(w \log \log \log n)$. Sobre este escenario podemos usar...

Pag. 45

CS3014 Estructuras de Datos Avanzadas Signature sort Paso 5 Ordenaremos las aristas del trie recursivamente usando (Indice del nodo, Primer digito, Indice del hijo). De esta manera, si el nivel actual tiene enteros dew' bits, estamos reduciendo a una tupla de $(O(\log n), O(w' \log \epsilon n), O(\log n))$, así que la cantidad de bits se reduce a $w' \log \epsilon n + O(\log n)$. Luego de 1 epsilon +1 niveles de recursion se tendra que la cantidad de bits de cada entero es $b = O(1 \epsilon \log n + w' \log(1 + \epsilon n)) = O(w \log \log \log n)$. Sobre este escenario podemos usar...

Pag. 46

CS3014 Estructuras de Datos Avanzadas Signature sort Paso 5 Ordenaremos las aristas del trie recursivamente usando (Indice del nodo, Primer digito, Indice del hijo). De esta manera, si el nivel actual tiene enteros dew' bits, estamos reduciendo a una tupla de $(O(\log n), O(w' \log \epsilon n), O(\log n))$, así que la cantidad de bits se reduce a $w' \log \epsilon n + O(\log n)$. Luego de 1 epsilon +1 niveles de recursion se tendra que la cantidad de bits de cada entero es $b = O(1 \epsilon \log n + w' \log(1 + \epsilon n)) = O(w \log \log \log n)$. Sobre este escenario podemos usar...

Pag. 47

CS3014 Estructuras de Datos Avanzadas Signature sort Paso 7 Ya que ahora tendremos los hijos de cada nodo del trie comprimido correctamente ordenados segun el valor original de sus datos, podemos ordenar calculando el inorder traversal del trie. En resumen 1. Dividimos cada entero en digitos para poder tener valores mas pequenos. 2. Aplicamos hashing para poder reducir el espacio usado para representar los digitos. 3. Usamos un trie comprimido para "ordenar" los hashes 4. Ordenamos recursivamente las aristas del trie comprimido para ordenarlos segun el...

Pag. 48

CS3014 Estructuras de Datos Avanzadas Signature sort Paso 7 Ya que ahora tendremos los hijos de cada nodo del trie comprimido correctamente ordenados segun el valor original de sus datos, podemos ordenar calculando el inorder traversal del trie. En resumen 1. Dividimos cada entero en digitos para poder tener valores mas pequenos. 2. Aplicamos hashing para poder reducir el espacio usado para representar los digitos. 3. Usamos un trie comprimido para "ordenar" los hashes 4. Ordenamos recursivamente las aristas del trie comprimido para ordenarlos segun el...

Pag. 49

CS3014 Estructuras de Datos Avanzadas Signature sort Paso 7 Ya que ahora tendremos los hijos de cada nodo del trie comprimido correctamente ordenados segun el valor original de sus datos, podemos ordenar calculando el inorder traversal del trie. En resumen 1. Dividimos cada entero en digitos para poder tener valores mas pequenos. 2. Aplicamos hashing para poder reducir el espacio usado para representar los digitos. 3. Usamos un trie comprimido para "ordenar" los hashes 4. Ordenamos recursivamente las aristas del trie comprimido para ordenarlos segun el...

Pag. 50

CS3014 Estructuras de Datos Avanzadas Signature sort Paso 7 Ya que ahora tendremos los hijos de cada nodo del trie comprimido correctamente ordenados segun el valor original de sus datos, podemos ordenar calculando el inorder traversal del trie. En resumen 1. Dividimos cada entero en digitos para poder tener valores mas pequenos. 2. Aplicamos hashing para poder reducir el espacio usado para representar los digitos. 3. Usamos un trie comprimido para "ordenar" los hashes 4. Ordenamos recursivamente las aristas del trie comprimido para ordenarlos segun el...

Pag. 51

CS3014 Estructuras de Datos Avanzadas Signature sort Paso 7 Ya que ahora tendremos los hijos de cada nodo del trie comprimido correctamente ordenados segun el valor original de sus datos, podemos ordenar

calculando el inorder traversal del trie. En resumen 1. Dividimos cada entero en digitos para poder tener valores mas pequenos. 2. Aplicamos hashing para poder reducir el espacio usado para representar los digitos. 3. Usamos un trie comprimido para "ordenar" los hashes 4. Ordenamos recursivamente las aristas del trie comprimido para ordenarlos segun el...

Pag. 52

CS3014 Estructuras de Datos Avanzadas Signature sort Paso 7 Ya que ahora tendremos los hijos de cada nodo del trie comprimido correctamente ordenados segun el valor original de sus datos, podemos ordenar calculando el inorder traversal del trie. En resumen 1. Dividimos cada entero en digitos para poder tener valores mas pequenos. 2. Aplicamos hashing para poder reducir el espacio usado para representar los digitos. 3. Usamos un trie comprimido para "ordenar" los hashes 4. Ordenamos recursivamente las aristas del trie comprimido para ordenarlos segun el...

Pag. 53

CS3014 Estructuras de Datos Avanzadas Signature sort Paso 7 Ya que ahora tendremos los hijos de cada nodo del trie comprimido correctamente ordenados segun el valor original de sus datos, podemos ordenar calculando el inorder traversal del trie. En resumen 1. Dividimos cada entero en digitos para poder tener valores mas pequenos. 2. Aplicamos hashing para poder reducir el espacio usado para representar los digitos. 3. Usamos un trie comprimido para "ordenar" los hashes 4. Ordenamos recursivamente las aristas del trie comprimido para ordenarlos segun el...

Pag. 54

CS3014 Estructuras de Datos Avanzadas Signature sort Paso 7 Ya que ahora tendremos los hijos de cada nodo del trie comprimido correctamente ordenados segun el valor original de sus datos, podemos ordenar calculando el inorder traversal del trie. En resumen 1. Dividimos cada entero en digitos para poder tener valores mas pequenos. 2. Aplicamos hashing para poder reducir el espacio usado para representar los digitos. 3. Usamos un trie comprimido para "ordenar" los hashes 4. Ordenamos recursivamente las aristas del trie comprimido para ordenarlos segun el...

Pag. 55

CS3014 Estructuras de Datos Avanzadas Resumen de la sesion -Aprendimos la equivalencia entre ordenamiento y colas de prioridad. -Aprendimos algunos resultados de ordenamientos de enteros. -Aprendimos signature sort. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 14 / 15

Pag. 56

CS3014 Estructuras de Datos Avanzadas Resumen de la sesion -Aprendimos la equivalencia entre ordenamiento y colas de prioridad. -Aprendimos algunos resultados de ordenamientos de enteros. -Aprendimos signature sort. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 14 / 15

Pag. 57

CS3014 Estructuras de Datos Avanzadas Resumen de la sesion -Aprendimos la equivalencia entre ordenamiento y colas de prioridad. -Aprendimos algunos resultados de ordenamientos de enteros. -Aprendimos signature sort. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 14 / 15

Pag. 58

CS3014 Estructuras de Datos Avanzadas Resumen de la sesion -Aprendimos la equivalencia entre ordenamiento y colas de prioridad. -Aprendimos algunos resultados de ordenamientos de enteros. -Aprendimos signature sort. Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 14 / 15

Pag. 59

CS3014 Estructuras de Datos Avanzadas Gracias Victor Racso Galvan Oyola Universidad de Ingenieria y Tecnologia 15 / 15

eda_slides_06_integer.pdf (48 paginas)

Pag. 1

Integer I CS3014 - Estructura de Datos Avanzados Luciano A. Romero Calla lromeroc@utec.edu.pe 2026-1

Pag. 2

Overview 1. Goals 2. Models 3. Predecesor problem 4. van Emde Boas 5. Tree view 6. Saving space 2/19

Pag. 3

Goals 1. Goals 3/19

Pag. 4

Goals Goals - Explore integer data structures to solve the predecessor problem. - Describe two main data structures van Emde Boas and Y-fast trees. 4/19

Pag. 5

Goals Goals - Explore integer data structures to solve the predecessor problem. - Describe two main data structures van Emde Boas and Y-fast trees. 4/19

Pag. 6

Models 2. Models 5/19

Pag. 7

Models Models word w -bit integer in $U = \{0, 1, 2, 3, \dots, u = 2^w - 1\}$ transdichotomous RAM bridging two worlds machine/problem, $w \geq \lg S$ word RAM transdichotomous RAM with C-style operations cell probe count # memory reads & writes, computation-free, useful for lower bounds cell probe strong transdichotomous RAM word RAM pointer machine BST weak 6/19

Pag. 8

Models Models word w -bit integer in $U = \{0, 1, 2, 3, \dots, u = 2^w - 1\}$ transdichotomous RAM bridging two worlds machine/problem, $w \geq \lg S$ word RAM transdichotomous RAM with C-style operations cell probe count # memory reads & writes, computation-free, useful for lower bounds cell probe strong transdichotomous RAM word RAM pointer machine BST weak 6/19

Pag. 9

Models Models word w -bit integer in $U = \{0, 1, 2, 3, \dots, u = 2^w - 1\}$ transdichotomous RAM bridging two worlds machine/problem, $w \geq \lg S$ word RAM transdichotomous RAM with C-style operations cell probe count # memory reads & writes, computation-free, useful for lower bounds cell probe strong transdichotomous RAM word RAM pointer machine BST weak 6/19

Pag. 10

Models Models word w -bit integer in $U = \{0, 1, 2, 3, \dots, u = 2^w - 1\}$ transdichotomous RAM bridging two worlds machine/problem, $w \geq \lg S$ word RAM transdichotomous RAM with C-style operations cell probe count # memory reads & writes, computation-free, useful for lower bounds cell probe strong transdichotomous RAM word RAM pointer machine BST weak 6/19

Pag. 11

Models Models word w -bit integer in $U = \{0, 1, 2, 3, \dots, u = 2^w - 1\}$ transdichotomous RAM bridging two worlds machine/problem, $w \geq \lg S$ word RAM transdichotomous RAM with C-style operations cell probe count # memory reads & writes, computation-free, useful for lower bounds cell probe strong transdichotomous RAM word RAM pointer machine BST weak 6/19

Pag. 12

Predecessor problem 3. Predecessor problem 7/19

Pag. 13

Predecessor problem Predecessor problem problem maintain a set of words - insert(x in U) - delete(x in S) - predecessor(x in U): $\max\{y \in S \mid y < x\}$ - successor(x in U): $\max\{y \in S \mid y > x\}$ BST comparison model: $O(\lg n)/op$ optimal word RAM - van Emde Boas: $O(\lg w)/op, \Theta(u)$ space - Y-fast trees: $O(\lg w)/op, \Theta(n)$ space - fusion trees: $O(\log w \cdot n)/op, \Theta(n)$ space cell probe lower bound $O(n \cdot \text{poly}(\lg n) \cdot \text{space})$, $\Omega(\min(\log w, n, \lg w \cdot \lg \lg n))$ pointer machine - word specified by node pointer - van Emde Boas: $O(\lg \lg u)/op, \Theta(u)$ space - lower bound: $\Omega(\lg \lg u)/op, \Omega(u)$ space...

Pag. 14

Predecessor problem Predecessor problem problem maintain a set of words - insert(x in U) - delete(x in S) - predecessor(x in U): $\max\{y \in S \mid y < x\}$ - successor(x in U): $\max\{y \in S \mid y > x\}$ BST comparison model: $O(\lg n)/op$ optimal word RAM - van Emde Boas: $O(\lg w)/op, \Theta(u)$ space - Y-fast trees: $O(\lg w)/op, \Theta(n)$ space - fusion trees: $O(\log w \cdot n)/op, \Theta(n)$ space cell probe lower bound $O(n \cdot \text{poly}(\lg n) \cdot \text{space})$, $\Omega(\min(\log w, n, \lg w \cdot \lg \lg n))$ pointer machine - word specified by node pointer - van Emde Boas: $O(\lg \lg u)/op, \Theta(u)$ space - lower bound: $\Omega(\lg \lg u)/op, \Omega(u)$ space...

Pag. 15

Predecessor problem Predecessor problem problem maintain a set Sofnwords - insert(x in U) - delete(x in S) - predecessor(x in U): $\max\{y \text{ in } S | y < x\}$ - successor(x in U): $\max\{y \text{ in } S | y > x\}$ BST comparison model: $O(\lg n)/\text{op}$ optimal word RAM - van Emde Boas: $O(\lg w)/\text{op}, \Theta(u)\text{space}$ - Y-fast trees: $O(\lg w)/\text{op}, \Theta(n)\text{space}$ - fusion trees: $O(\log w \lg n)/\text{op}, \Theta(n)\text{space}$ cell probe lower bound $O(n \text{ poly} \lg n)\text{space}, \Omega(\min \log w, n, \lg w \lg \lg n)$ pointer machine - word specified by node pointer - van Emde Boas: $O(\lg \lg u)/\text{op}, \Theta(u)\text{space}$ - lower bound: $\Omega(\lg \lg u)/\text{op}, \Omega(u)\text{spa}...$

Pag. 16

Predecessor problem Predecessor problem problem maintain a set Sofnwords - insert(x in U) - delete(x in S) - predecessor(x in U): $\max\{y \text{ in } S | y < x\}$ - successor(x in U): $\max\{y \text{ in } S | y > x\}$ BST comparison model: $O(\lg n)/\text{op}$ optimal word RAM - van Emde Boas: $O(\lg w)/\text{op}, \Theta(u)\text{space}$ - Y-fast trees: $O(\lg w)/\text{op}, \Theta(n)\text{space}$ - fusion trees: $O(\log w \lg n)/\text{op}, \Theta(n)\text{space}$ cell probe lower bound $O(n \text{ poly} \lg n)\text{space}, \Omega(\min \log w, n, \lg w \lg \lg n)$ pointer machine - word specified by node pointer - van Emde Boas: $O(\lg \lg u)/\text{op}, \Theta(u)\text{space}$ - lower bound: $\Omega(\lg \lg u)/\text{op}, \Omega(u)\text{spa}...$

Pag. 17

Predecessor problem Predecessor problem problem maintain a set Sofnwords - insert(x in U) - delete(x in S) - predecessor(x in U): $\max\{y \text{ in } S | y < x\}$ - successor(x in U): $\max\{y \text{ in } S | y > x\}$ BST comparison model: $O(\lg n)/\text{op}$ optimal word RAM - van Emde Boas: $O(\lg w)/\text{op}, \Theta(u)\text{space}$ - Y-fast trees: $O(\lg w)/\text{op}, \Theta(n)\text{space}$ - fusion trees: $O(\log w \lg n)/\text{op}, \Theta(n)\text{space}$ cell probe lower bound $O(n \text{ poly} \lg n)\text{space}, \Omega(\min \log w, n, \lg w \lg \lg n)$ pointer machine - word specified by node pointer - van Emde Boas: $O(\lg \lg u)/\text{op}, \Theta(u)\text{space}$ - lower bound: $\Omega(\lg \lg u)/\text{op}, \Omega(u)\text{spa}...$

Pag. 18

van Emde Boas 4. van Emde Boas 4.1 Definition 4.2 Operations 9/19

Pag. 19

van Emde Boas Definition Definition idea $T(u) = T(\sqrt{u}) + O(1)$ hierarchical coordinates word $x = c, i$ recursive vEBV of size u - $V.\text{cluster}[i] = \text{vEB}$ of size $\sqrt{u}$ - $V.\text{summary}[i] = \text{vEB}$ of size $\sqrt{u}$ - $V.\text{min} = \text{minimum element in } V$ - $V.\text{max} = \text{maximum element in } V$ 10/19

Pag. 20

van Emde Boas Definition Definition idea $T(u) = T(\sqrt{u}) + O(1)$ hierarchical coordinates word $x = c, i$ recursive vEBV of size u - $V.\text{cluster}[i] = \text{vEB}$ of size $\sqrt{u}$ - $V.\text{summary}[i] = \text{vEB}$ of size $\sqrt{u}$ - $V.\text{min} = \text{minimum element in } V$ - $V.\text{max} = \text{maximum element in } V$ 10/19

Pag. 21

van Emde Boas Definition Definition idea $T(u) = T(\sqrt{u}) + O(1)$ hierarchical coordinates word $x = c, i$ recursive vEBV of size u - $V.\text{cluster}[i] = \text{vEB}$ of size $\sqrt{u}$ - $V.\text{summary}[i] = \text{vEB}$ of size $\sqrt{u}$ - $V.\text{min} = \text{minimum element in } V$ - $V.\text{max} = \text{maximum element in } V$ 10/19

Pag. 22

van Emde Boas Definition Definition idea $T(u) = T(\sqrt{u}) + O(1)$ hierarchical coordinates word $x = c, i$ recursive vEBV of size u - $V.\text{cluster}[i] = \text{vEB}$ of size $\sqrt{u}$ - $V.\text{summary}[i] = \text{vEB}$ of size $\sqrt{u}$ - $V.\text{min} = \text{minimum element in } V$ - $V.\text{max} = \text{maximum element in } V$ 10/19

Pag. 23

van Emde Boas Definition Definition idea $T(u) = T(\sqrt{u}) + O(1)$ hierarchical coordinates word $x = c, i$ recursive vEBV of size u - $V.\text{cluster}[i] = \text{vEB}$ of size $\sqrt{u}$ - $V.\text{summary}[i] = \text{vEB}$ of size $\sqrt{u}$ - $V.\text{min} = \text{minimum element in } V$ - $V.\text{max} = \text{maximum element in } V$ 10/19

Pag. 24

van Emde Boas Definition Definition idea $T(u) = T(\sqrt{u}) + O(1)$ hierarchical coordinates word $x = c, i$ recursive vEBV of size u - $V.\text{cluster}[i] = \text{vEB}$ of size $\sqrt{u}$ - $V.\text{summary}[i] = \text{vEB}$ of size $\sqrt{u}$ - $V.\text{min} = \text{minimum element in } V$ - $V.\text{max} = \text{maximum element in } V$ 10/19

Pag. 25

van Emde Boas Definition Definition Figure 1: van Emde Boas structure 1 1 Demaine 2021. 11/19

Pag. 26

van Emde Boas Operations Operations Successor($V, x = c, i$) - if $x < V.min$ return $V.min$ - if $i < V.cluster[c].max$ return $c, Successor(V.cluster[c], i)$ - else $c' = Successor(V.summary, c)$ return $c', V.cluster[c'] .min$ 12/19

Pag. 27

Tree view 5. Tree view 5.1 Indirection 13/19

Pag. 28

Tree view Tree view Figure 2: van Emde Boas tree view 2 2 Demaine 2021. 14/19

Pag. 29

Tree view Indirection Indirection - from $O(\lg w)$ query, $O(w)$ update DS - reduce to $O(\lg w)$ update: split elements into chunks of size $\Theta(w)$ 15/19

Pag. 30

Tree view Indirection Indirection - from $O(\lg w)$ query, $O(w)$ update DS - reduce to $O(\lg w)$ update: split elements into chunks of size $\Theta(w)$ 15/19

Pag. 31

Saving space 6. Saving space 6.1 X-fast trees 6.2 Y-fast trees 16/19

Pag. 32

Saving space Saving space - don't store empty clusters vEB - $V.clusters =$ hash table - charge each table entry to min in child - space bound $\Theta(n)$ via indirection 17/19

Pag. 33

Saving space Saving space - don't store empty clusters vEB - $V.clusters =$ hash table - charge each table entry to min in child - space bound $\Theta(n)$ via indirection 17/19

Pag. 34

Saving space Saving space - don't store empty clusters vEB - $V.clusters =$ hash table - charge each table entry to min in child - space bound $\Theta(n)$ via indirection 17/19

Pag. 35

Saving space Saving space - don't store empty clusters vEB - $V.clusters =$ hash table - charge each table entry to min in child - space bound $\Theta(n)$ via indirection 17/19

Pag. 36

Saving space X-fast trees X-fast trees - don't store 0s - store a hash table of root-to-1 path - $O(\lg w)$ query binary search - $\Theta(w)$ update - $\Theta(nw)$ space 18/19

Pag. 37

Saving space X-fast trees X-fast trees - don't store 0s - store a hash table of root-to-1 path - $O(\lg w)$ query binary search - $\Theta(w)$ update - $\Theta(nw)$ space 18/19

Pag. 38

Saving space X-fast trees X-fast trees - don't store 0s - store a hash table of root-to-1 path - $O(\lg w)$ query binary search - $\Theta(w)$ update - $\Theta(nw)$ space 18/19

Pag. 39

Saving space X-fast trees X-fast trees - don't store 0s - store a hash table of root-to-1 path - $O(\lg w)$ query binary search - $\Theta(w)$ update - $\Theta(nw)$ space 18/19

Pag. 40

Saving space X-fast trees X-fast trees - don't store 0s - store a hash table of root-to-1 path - $O(\lg w)$ query binary search - $\Theta(w)$ update - $\Theta(nw)$ space 18/19

Pag. 41

Saving space Y-fast trees Y-fast trees - = X-fast trees - + indirection - $O(\lg w)$ query binary search - $O(\lg w)$ update amortized - $O(\lg w)$ space 19/19

Pag. 42

Saving space Y-fast trees Y-fast trees - = X-fast trees - + indirection - $O(\lg w)$ query binary search - $O(\lg w)$ update amortized - $O(\lg w)$ space 19/19

Pag. 43

Saving space Y-fast trees Y-fast trees - = X-fast trees - + indirection - $O(\lg w)$ query binary search - $O(\lg w)$ update amortized - $O(\lg w)$ space 19/19

Pag. 44

Saving space Y-fast trees Y-fast trees - = X-fast trees - + indirection - $O(\lg w)$ query binary search - $O(\lg w)$ update amortized - $O(\lg w)$ space 19/19

Pag. 45

Saving space Y-fast trees Y-fast trees - = X-fast trees - + indirection - $O(\lg w)$ query binary search - $O(\lg w)$ update amortized - $O(\lg w)$ space 19/19

Pag. 46

Thanks

Pag. 47

References References Demaine, Erik (2021).6.851: Advanced Data Structures (Spring'21).url: <https://courses.csail.mit.edu/6.851/spring21/>. 21/19

Pag. 48

Acknowledgements Acknowledgements The course slides are based on the lectures from previous editions and on similar lectures elsewhere. List of credits: Erick Demaine, Keith Schwarz 22/19

Cierre: si tienes poco tiempo

Prioriza así:

1. Modelos: word RAM, AC0 RAM, cell probe, universo $u = 2^w$.
2. Predecessor/successor: definicion exacta y por que BST es $O(\log n)$.
3. vEB: clusters, summary, min/max, recurrence $O(\log \log u)$.
4. X-fast/Y-fast: prefijos hash e indirection para espacio lineal.
5. Fusion Trees: B-tree de grado $w^{1/5}$, sketches y $O(\log_w n)$.
6. Counting/radix sort: condiciones para linealidad.
7. Signature sort: historia conceptual, no implementacion.

Para una respuesta de examen, casi siempre basta con explicar el invariante, la operacion principal y la complejidad. No escribas formulas avanzadas si no las puedes conectar con el problema.